

Semantic Analysis

Slides based on material
by Ras Bodik available at
<http://inst.eecs.berkeley.edu/~cs164/fa04>

Symbol Table

Outline

- How to build symbol tables
- How to use them to find
 - multiply-declared and
 - undeclared variables.

Errori di
Dichiarazione

The Compiler So Far

Front-End
Compiler

- **Lexical analysis** Lexer
 - Detects inputs with illegal tokens
 - e.g.: main£ ();
- **Parsing** Parser
 - Detects inputs with ill-formed parse trees
 - e.g.: missing semicolons
- **Semantic analysis**
 - Last “front end” phase
 - Catches all remaining errors

Introduction

L'Analisi Semantica agisce come un "controllore del significato" subito dopo l'Analisi Sintattica (che controlla la grammatica). Mentre la sintassi verifica se le parole sono disposte correttamente, la semantica verifica se le parole hanno senso nel contesto del linguaggio e del programma.

- typical semantic errors:

Tenta di dichiarare la stessa entità (variabile, funzione, classe, ecc.) più di una volta nello stesso ambito (scope).

multiple declarations: a variable should be declared (in the same scope) at most once

Perché è un errore: Genera ambiguità per il compilatore. Se dichiarare `int x;` e poi di nuovo `float x;` nello stesso blocco di codice, il compilatore non sa quale tipo o quale `x` usare in seguito.

Tenta di utilizzare un identificatore che non è stato precedentemente introdotto nell'ambito corrente o in un ambito accessibile.

undeclared variable: a variable should not be used before being declared.

Perché è un errore: Il compilatore non può assegnare una locazione di memoria o un tipo a un identificatore sconosciuto. In termini tecnici, non trova un'associazione per l'identificatore nella tabella dei simboli.

Un'operazione (ad esempio, un'assegnazione o un'operazione aritmetica) viene eseguita su operandi di tipi incompatibili che non possono essere convertiti automaticamente (coercizione).

type mismatch: e.g. type of the left-hand side of an assignment should match the type of the right-hand side.

Una funzione o un metodo viene chiamato con un numero di parametri effettivi diverso dal numero di parametri formali attesi, oppure i tipi dei parametri effettivi non corrispondono ai tipi attesi (o non sono compatibili).

wrong argument number/types: methods should be called with the right number and types of arguments.

Perché è un errore: La funzione è stata progettata per lavorare con un set specifico di dati. Fornire dati diversi o in quantità diversa impedisce l'esecuzione logica della funzione.



La seconda fase si svolge dopo che tutti i nomi sono stati risolti e l'AST è stato arricchito con i puntatori.

- Visita Bottom-Up: Inizia dalle foglie dell'AST e risale verso la radice. Questo è fondamentale per determinare il tipo di espressioni complesse.
- Uso del Puntatore alla Tabella dei Simboli: Per ogni nodo ID nell'AST, il compilatore usa il puntatore allegato durante la Fase 1 per recuperare il tipo di quella variabile.
- Verifica degli Errori di Tipo: In corrispondenza dei nodi operatore (come +, =, o chiamate di funzione), il compilatore:
 - Determina il Tipo dell'Espressione: Basandosi sui tipi degli operandi (ottenuti tramite l'analisi dei nodi figli), calcola il tipo risultante dell'operazione.
 - Cerca Disallineamenti: Verifica se l'operazione è valida per quei tipi o se i tipi in un'assegnazione sono compatibili. Se non lo sono, segnala l'errore semantico di "type mismatch".

A simple semantic analyzer

Symbol Table = mantiene le entry delle dichiarazioni dei nomi, che vengono collegate agli usi di tali nomi

Siamo in un linguaggio tipizzato staticamente

works in two phases

- ^{visitando} by visiting the AST created by the parser

Obiettivo della Fase 1: Risolvere i nomi ed assicurare che tutte le entità utilizzate siano state dichiarate in modo univoco e accessibile.

1. Generation of Enriched AST (performed top-down)

For each scope in the program:

process the declarations =

- add new entries to the **symbol table** and
- report any variable/method that is **multiply declared**

process the statements =

- find uses of **undeclared variables**, and
- in **"ID" nodes** (variable/method **usages**) of the AST add a pointer to the appropriate **symbol table entry**.

2. Type Checking (performed bottom-up) *

Process all of the statements in the program again,

- use **attached symbol-table entry information** to determine the **type of each expression**, and to find type errors.

Quando la visita raggiunge il codice che usa le variabili:

- Rilevamento di Variabili Non Dichiarate: Se incontra un nodo ID (identificatore, un uso di variabile/metodo) e non riesce a trovarlo in nessuno degli scope visibili nella Tabella dei Simboli, segnala l'errore semantico di "variabile non dichiarata".
- Arricchimento dell'AST (Linking): Se l'identificatore viene trovato, il nodo ID nell'AST viene arricchito aggiungendo un puntatore diretto all'entry corrispondente nella Tabella dei Simboli.

Questa prima fase è focalizzata sulla gestione dello scope e sulla costruzione del contesto (la Tabella dei Simboli).

- Il compilatore visita l'AST dall'alto verso il basso.

- Aggiunta alla Tabella dei Simboli: Quando incontra una dichiarazione, crea una nuova entry nella Tabella dei Simboli. Questa entry contiene nome, tipo e scope.

- Rilevamento di Dichiarazioni Multiple: Se, tentando di aggiungere una nuova entry, scopre che un identificatore con lo stesso nome esiste già nello stesso scope, segnala immediatamente l'errore semantico di "dichiarazione multipla".

Obiettivo della Fase 2: Verificare la coerenza dei tipi e annotare ciascun nodo dell'espressione nell'AST con il suo tipo calcolato. Questo è cruciale per le fasi successive del compilatore.

Symbol Table = map from names to entries

- **purpose:**
 - keep track of names declared in the program
 - names of
 - variables, classes, fields, methods, ETC....
- **symbol table entry:** informazioni sul nome
 - set of attributes associated with a name, per esempio e.g.:
 - kind of name (variable, class, field, method, etc)
 - type (int, float, etc)
 - nesting level
 - memory location (i.e., where it will be found at runtime).

Scoping

→ Il ST può cambiare in base al tipo di scoping usato dal linguaggio di programmazione

- symbol table design influenced by what kind of **scoping** is used by the compiled language
- In most languages, the same name can be **declared multiple times**
 - if its declarations occur in **different scopes**, and/or
 - involve **different kinds of names**.

Scoping: example

- Java: can use same name for
 - a class,
 - field of the class,
 - a method of the class, and
 - a local variable of the method
- *legal Java program:*

```
class Test {  
    int Test;  
    void Test( ) { double Test; }  
}
```

Il nome é costituito da nome metodo
+ tipo dei parametri (signature)

Quando il compilatore incontra una chiamata `sum(a, b)`, non gli basta trovare il nome `sum`. Deve fare un'ulteriore verifica:

- Determina il tipo degli argomenti effettivi (`a` e `b`) (questo richiede in realtà un'anticipazione del Type Checking della Fase 2).
- Cerca nella Tabella dei Simboli l'entry per `sum` che corrisponde esattamente o è compatibile con i tipi degli argomenti forniti.

Scoping: overloading

- **Java and C++** (but not in Pascal or C):
 - can use the same name for more than one method
 - as long as the number and/or types of parameters are unique.

Numero di parametri (es. `sum(int a)` vs `sum(int a, int b)`).
Tipo dei parametri (es. `print(int x)` vs `print(String s)`).

`int add(int a, int b);`

`float add(float a, float b);`

Le regole di scope di un linguaggio sono fondamentali per l'Analisi Semantica perché stabiliscono la visibilità degli identificatori (variabili, funzioni, classi) e risolvono l'ambiguità.

Funzione Principale: Le regole determinano, per ogni utilizzo di un elemento con nome (es. x), quale specifica dichiarazione di quell'elemento si intende. In termini di compilatore, mappano l'uso alla sua dichiarazione nella Tabella dei Simboli.

Scoping: general rules

- The scope rules of a language:
 - determine which declaration of a named element (e.g. a variable) corresponds to each use of the element.
 - i.e., scoping rules map uses of element to their declarations.

- C++ and Java use *static scoping*:
 - Si differenzia dallo Scoping Dinamico, dove la mappatura dipende dalla sequenza di chiamate di funzione a runtime.

- mapping from uses to declarations is made at compile time.
- C++ uses the "most closely nested" rule

Viene scelta la dichiarazione trovata nello scope più vicino e che precede testualmente il punto d'uso.

- a use of variable x matches the declaration in the **most closely enclosing scope** such that the declaration precedes the use.
- inner scope variable x declaration **hides** x declared in an outer scope.
- in Java:
 - inner scopes cannot define variables defined in outer scopes

Scope levels

- Each function has one or more scopes:
 - one for the parameters and the function body,
 - and possibly additional scopes in the function
 - each nested block (delimited by curly braces)

Example (assume C++ rules)

```
void f( int y ) {  
    int k = 0;  
    while (....) {  
        int k = 1;  
    }  
}
```

Outmost
scope che
include in esso
la funzione f

// y is a parameter
// k is a local variable

// another local var, in a loop (not ok in Java)

- ⊖ the outmost scope includes just the name "f", and
- function f itself has two inner (nested) scopes:
 - ① The scope for the body of f, which includes parameter y and local variable k that is initialized to 0.
 - ② The innermost scope is for the body of the while loop, and includes the variable k that is initialized to 1.

TEST YOURSELF

- This is a C++ program. Match each use to its declaration, or say why it is a use of an undeclared variable.

```
class Foo {  
    int k=10, x=20;  
    void foo(int k) {  
        int a = x;  
        int x = k;  
        int b = x;  
        while (...) {  
            int x=11;  
            if (x == k) {  
                int k, y;  
                k = (y = x);  
            }  
            if (x == k) { int x, y; }  
        }  
    }  
}
```

Questo k nella funzione foo, non é lo stesso dichiarato nello scope della classe Foo

Lo scoping dinamico è un modello di risoluzione dei nomi in cui un identificatore (variabile) viene associato alla sua dichiarazione in base alla sequenza di chiamate di funzione attive (lo stack di esecuzione), non alla posizione del codice nel file sorgente.

Dynamic scoping

Storicamente, si usano perché molto semplice da implementare rispetto al Static Scoping

- Not all languages use static scoping.
- Lisp, APL, and Snobol use dynamic scoping.
- Other languages, like Common Lisp or Perl, allow the programmer to choose among static or dynamic scoping
- Dynamic scoping:
 - Use of a variable with no corresponding declaration in the same function (non-local reference)
 - corresponds to the declaration in most-recently-called still active function.



La regola che governa lo Scoping Dinamico è la seguente: Per un uso di variabile non locale (cioè, quando la variabile non è dichiarata nella funzione corrente), la dichiarazione corrispondente è quella presente nella funzione ancora attiva (nello stack) che è stata chiamata più di recente.

Significato per la Tabella dei Simboli (Run-time): Quando un'uso di variabile non è risolto localmente, il compilatore cerca la sua dichiarazione risalendo lo stack di chiamate (stack trace). Si ferma alla prima dichiarazione trovata per quell'identificatore.

Example

- For example, consider the following code:

```
char x;  
void main() { f1(); f2(); }  
void f1() { int x = 10; g(); }  
void f2() { String x = "hello"; f3(); g(); }  
void f3() { double x = 30.5; }  
void g() { print(x); }
```

Con le chiamate, main -> f1 -> g; x = 10

Con le chiamate,
main -> f2 -> g;
x='hello'

Scoping Statico: x si riferisce alla dichiarazione 'char x'

Scoping Dinamico: x dipende dalla sequenza delle chiamate di funzioni che rimangono attive

'hello' perché f3
quando viene
chiamata g, f3 è
già terminata,
quindi non è
presente nello
stack trace delle
chiamate

TEST YOURSELF

Con Scoping Dinamico, guardo lo stack trace delle chiamate delle funzioni ancora attive
Con Scoping Statico, guardo lo scope

- Assuming that static (resp. dynamic) scoping is used, what is output by the following program?

```
{ int x = 0;  
  void f() {int x=1; g();}  
  void g() {print x;  
           f(); }
```

Scoping Statico: Stampa 0
Scoping Dinamico: Stampa 1

Static vs dynamic scoping

- generally, dynamic scoping is a bad idea
 - can make a program difficult to understand
 - a single use of a variable can correspond to
 - many different declarations
 - with different types!

Used before declared?

I campi e metodi di un oggetto di una classe vengono allocati nell'Heap insieme all'oggetto a cui fanno parte, mentre le variabili nello stack

- can a name be used before it is defined?
 - Java: a method or field name can be used before the definition appears,
 - not true for a variable!

Example JAVA

```
class Test {  
    void f() {  
        val = 0;  
        // field val has not yet been declared -- OK  
        g();  
        // method g has not yet been declared -- OK  
        x = 1;  
        // var x has not yet been declared -- ERROR!  
        int x;  
    }  
    void g() {}  
    int val;  
}
```

Simplification

Applichiamo delle semplificazioni per capire meglio l'analisi semantica

- From now on, assume that our language:
 - 1 - uses static scoping
 - 2 - requires that all names be declared before they are used
 - 3 - does not allow multiple declarations of a name in the same scope
 - e.g. no method overloading
 - even for different kinds of names
 - e.g. field and method with the same name not allowed
 - 4 - does allow the same name to be declared in multiple nested scopes (simile a C++)
 - but only once per scope

Symbol Table Implementations

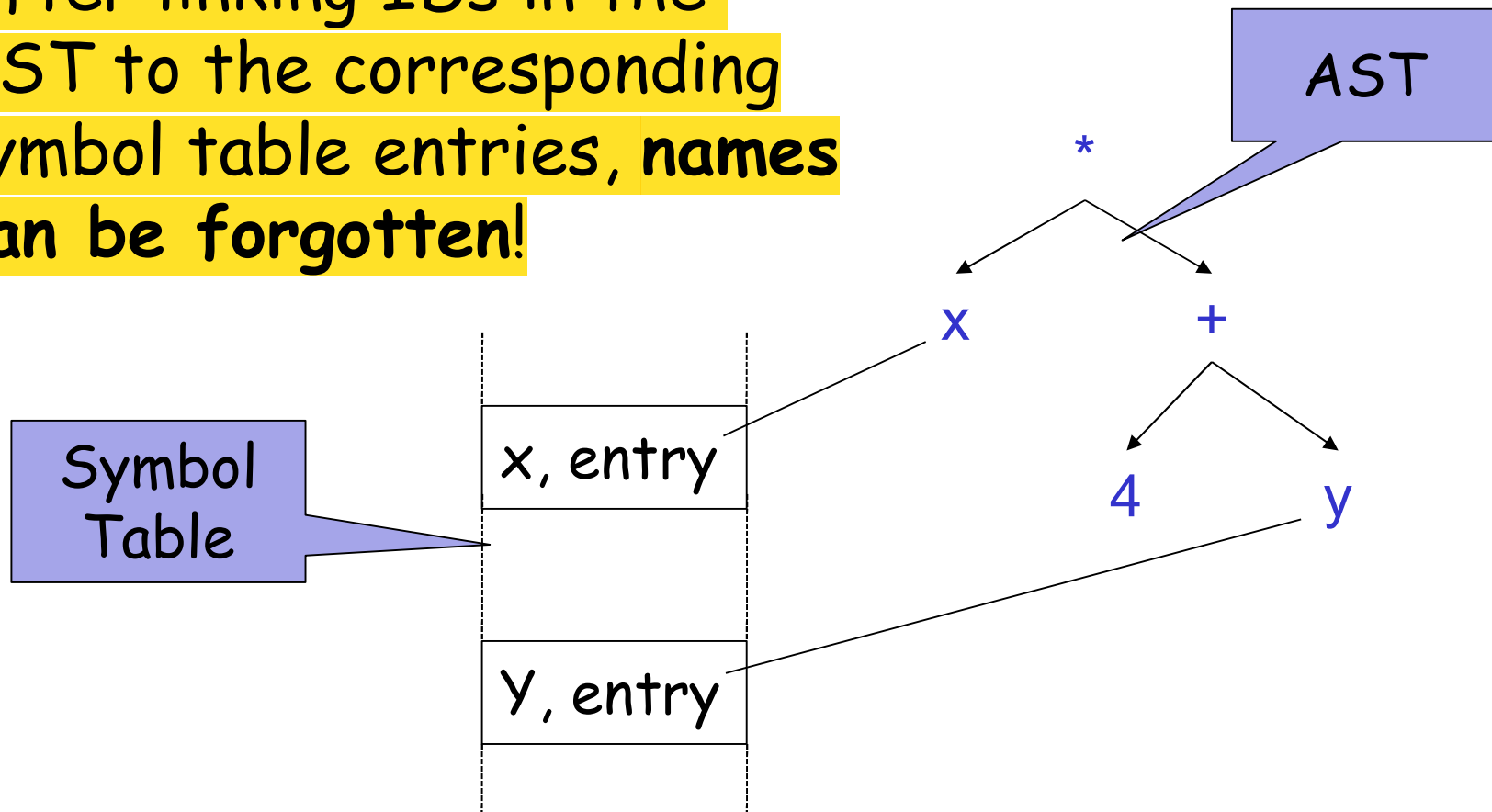
- In addition to the above simplification, assume that the symbol table will be used to answer two questions:
 1. Given a declaration of a name, is there already a declaration of the same name in the current scope
 - i.e., is it multiply declared?
 2. Given a use of a name, to which declaration does it correspond (using the "most closely nested" rule), or is it undeclared?

Note

- The symbol table is only needed to answer those two questions, i.e.
 - 1 - once all declarations have been processed to build the symbol table,
 - 2 - and all uses have been processed to link each ID node in the abstract-syntax tree with the corresponding symbol-table entry,
- then the symbol table itself is no longer needed
 - because no more lookups based on name will be performed

Graphically...

- After linking IDs in the AST to the corresponding symbol table entries, names can be forgotten!



What operation do we need?

- Given the above assumptions, we will need:

- (for name declarations)

Se la ricerca ha successo, si è verificato un errore di dichiarazione multipla (un errore semantico)

Look up a name in the current scope only

Cerca l'identificatore (name) SOLO nell'ambito di visibilità corrente.

- to check if it is multiply declared

and insert a new name into the symbol table mapped to an entry containing its attributes.

Inserisce il nuovo identificatore nella Tabella dei Simboli, associandolo ai suoi attributi

- (for name usages)

Look up a name in the current and enclosing scopes

Cerca l'identificatore, partendo dallo scope corrente e risalendo in modo gerarchico tutti gli scope annidati esterni (enclosing scopes). Si ferma alla prima corrispondenza trovata

- to check for a use of an undeclared name, and
- to link a use with the corresponding symbol-table entry.

- Do what must be done when a new scope is entered.

Crea un nuovo scope nella struttura dati della Tabella dei Simboli.

- Do what must be done when a scope is exited.

Rimuove l'ambito corrente, rendendo inaccessibili (e liberabili in memoria) tutte le dichiarazioni locali che conteneva. Questo è fondamentale per prevenire l'accesso errato a variabili fuori scope.

Two possible symbol table implementations

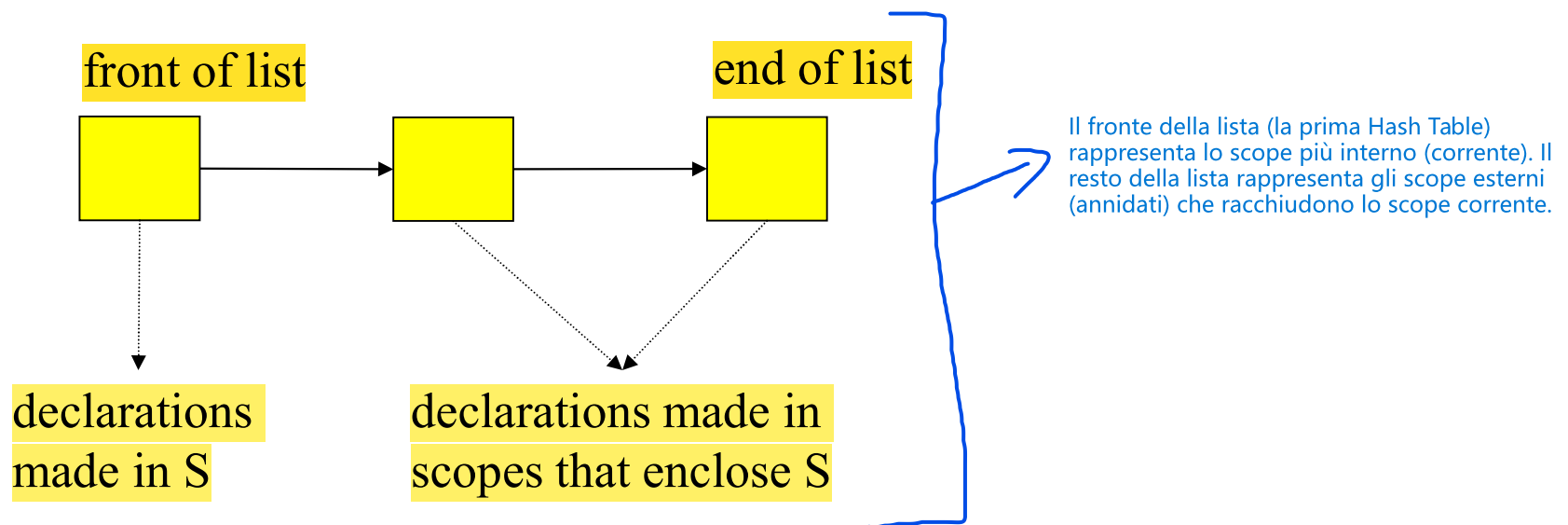
1. a list of tables Lista di Tabelle
 2. a table of lists Tabella di Liste
- For each approach, we will consider
 - what must be done when processing a declaration,
 - when processing a use, and
 - when entering and exiting a scope.
 - Simplification:
 - assume each symbol-table entry includes only:
 - its type and the nesting level of its declaration

Method 1: List of Hashtables

L'idea centrale di questa implementazione è quella di rappresentare la gerarchia degli scope attivi utilizzando una lista, dove ogni nodo della lista è un'hash table dedicata a un singolo scope.

- The idea:
 - symbol table = a list of hashtables,
 - one hashtable for each currently visible scope.
- When processing a scope S:

→ Ogni singola Hash Table nella lista memorizza tutte le dichiarazioni (variabili, funzioni, classi) che appartengono esclusivamente a un determinato scope attualmente visibile.

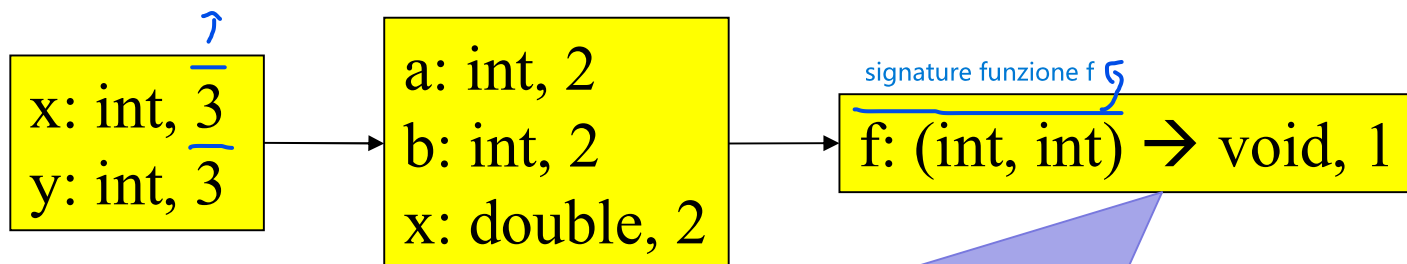


Example:

Alla creazione di una nuova hashtable, viene incrementata la variabile globale che mantiene il conto del nesting level in cui si é attualmente

```
void f(int a, int b) {  
    double x;  
    while (...) { int x, y; ... }  
}  
void g() { f(); }
```

- After processing declarations inside the while loop:



Method type:
function with domain → codomain

List of hashtables: the operations

1. On scope entry:

Entrata nello scope

- increment the current level number and add a new empty hashtable to the front of the list.

2. To process a declaration of x:

Processa una dichiarazione x

- look up x in the first table in the list.
 - If it is there, then issue a "multiply declared variable" error;
 - otherwise, add x to the first table in the list mapped to a new entry containing x type and nesting level.

... continued

3. To process a use of x: Processa un uso di x

- look up x starting in the first table in the list;
 - if it is not there, then look up x in each successive table in the list.
 - link the use of x with the found symbol-table entry
 - If, instead, x is not in any table then issue an "undeclared variable" error.

4. On scope exit, Uscita da scope

- remove the first table from the list and decrement the current level number.

Remember

il nome di un metodo appartiene allo scope esterno (contenitore), non allo scope interno del metodo stesso.

- method names belong to the hashtable for the outer scope (w.r.t. inner method scopes)
 - i.e. not to the same table as the method's variables/parameters
- For instance, in the example above:
 - method name f is in the symbol table for the outermost scope
 - name f is not in the same scope as parameters a and b, and variable x.
 - This is so that when the use of name f in method g is processed, the name is found in an enclosing scope's table.

The running times for each operation:

1. **Scope entry:** Entrare in un nuovo scope richiede di creare una nuova, vuota hash table e di aggiungerla in cima alla lista (o allo stack) dei scope attivi.
 - time to initialize a new, empty hashtable;
 - probably proportional to the size of the hashtable.
2. **Process a declaration:** L'inserimento di un nuovo simbolo avviene solo nella Hash Table dello scope corrente (quella in testa alla lista).
 - using hashing, constant expected time ($O(1)$).
3. **Process a use:** Questa è l'operazione più variabile. L'Analisi Semantica deve risalire la catena degli scope (la lista di Hash Table) fino a trovare la dichiarazione
 - using hashing to do the lookup in each table in the list, the worst-case time is $O(\text{depth of nesting})$, when every table in the list must be examined.
4. **Scope exit:** Uscire da uno scope richiede di rimuovere la Hash Table in testa alla lista.
 - time to remove a table from the list, which should be $O(1)$



Punto Debole: complessità temporale proporzionale alla profondità della lista

TEST YOURSELF

- Assume that the symbol table is implemented using a list of hashtables.
- Draw pictures to show how the symbol table changes as each declaration in the following code is processed.

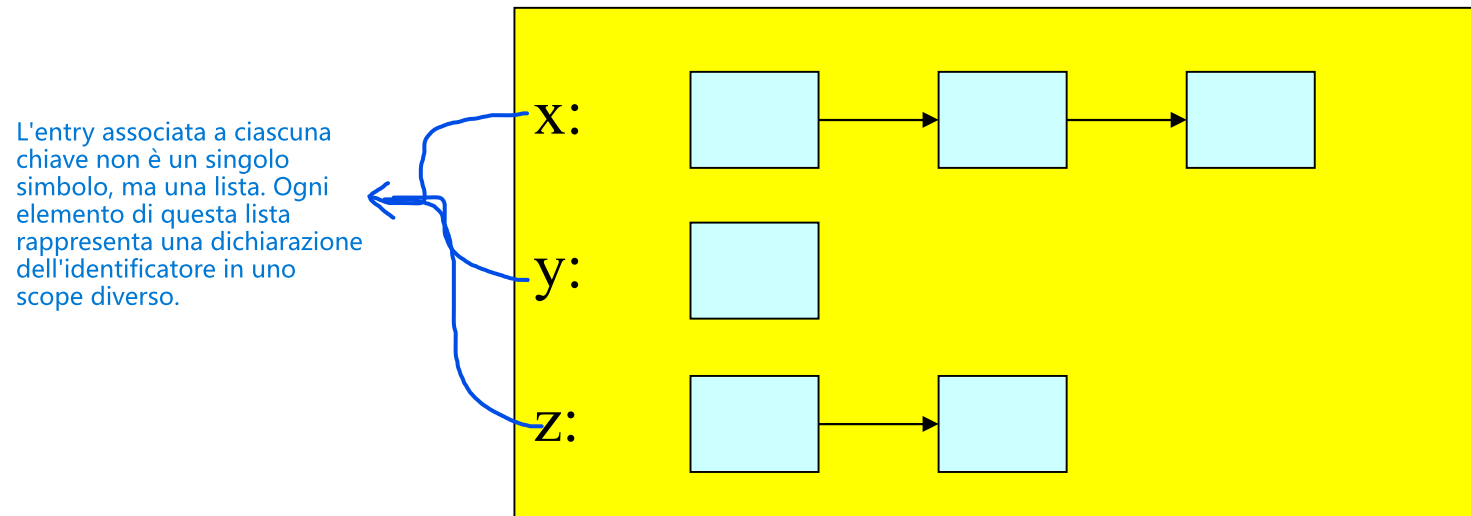
```
void g(int x, int a) {  
    double d;  
    while (...) {  
        int d, w;  
        double x, b;  
        if (...) { int a,b,c; }  
    }  
    while (...) { int x,y,z; }  
}
```

In questo approccio, si utilizza un'unica Hash Table per l'intero programma, e la gestione dello scoping annidato (l'ombreggiamento delle variabili in scope esterni) viene demandata alle liste concatenate associate a ciascun identificatore.

Method 2: Hashtable of Lists

Un'unica tabella di Liste

- the idea:
 - when processing a scope S , the structure of the symbol table is:



Ordine delle Liste: Per implementare la regola del "most closely nested" (scope più interno), le dichiarazioni vengono aggiunte alla testa della lista (come uno stack).

Definition

Esiste una singola Hash Table che funge da struttura di base per l'intera Tabella dei Simboli. Questa tabella contiene una chiave per ogni identificatore (nome di variabile, funzione, ecc.) che è attualmente visibile nello scope corrente o in qualsiasi scope annidato esterno che lo racchiude.

there is just one big hashtable, containing an entry for each name for which there is

- some declaration in scope *S* or
- in a scope that encloses *S*.

Associated with each name is a list of symbol-table entries.

- The first list item corresponds to the most closely enclosing declaration;
- the other list items correspond to declarations in enclosing scopes.

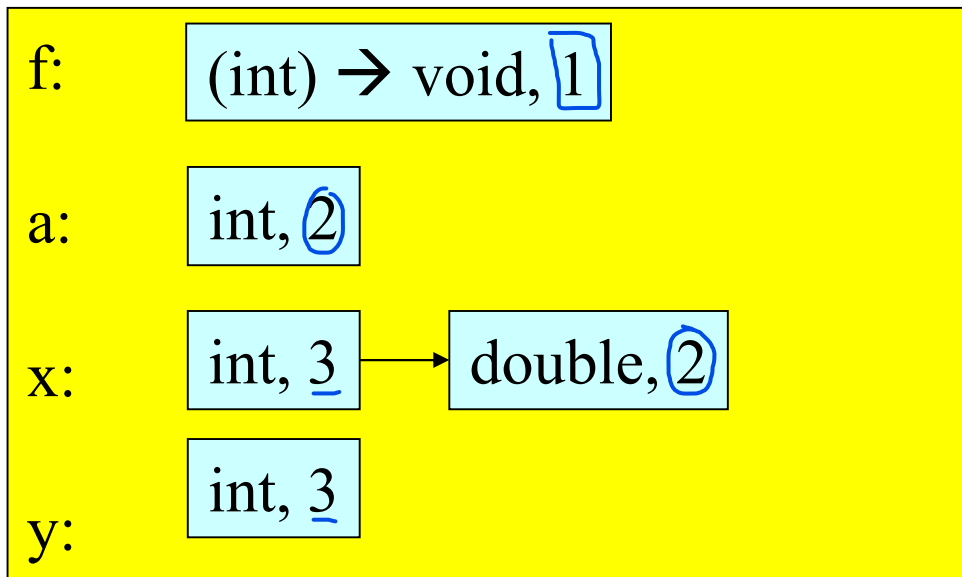
- Lista di Entry: Ad ogni nome nella Hash Table è associata una lista concatenata di Symbol Table Entries (STE). Ogni STE in questa lista rappresenta una dichiarazione di quel nome.
- Ordine Cruciale: L'ordine in cui sono disposte le dichiarazioni nella lista è fondamentale per implementare la regola del "most closely nested" (la più strettamente annidata):
> Testa della Lista: Il primo elemento della lista (quello in testa) corrisponde alla dichiarazione fatta nello scope più interno (più vicino all'uso corrente).
> Elementi Successivi: Gli altri elementi della lista rappresentano le dichiarazioni dello stesso nome che si trovano negli scope via via più esterni che racchiudono lo scope corrente.

Example

All'accesso in uno scope innestato, viene incrementata la variabile globale che mantiene il conto del nesting level in cui si é attualmente

```
void f(int a) {  
    double x;  
    while (...) { int x, y; ... }  
}  
void g() { f(); }
```

- After processing the declarations inside the while loop:



Nesting level information is crucial

- the level-number attribute stored in each list item enables us to determine whether the most closely enclosing declaration was made
 - in the current scope or
 - in an esterno enclosing scope.

Hashtable of lists: the operations

1. On scope entry: Entrata in uno scope

- increment the current level number. (incrementa la variabile globale che tiene conto del livello di scope in cui siamo)

2. To process a declaration of x: Processa la dichiarazione di x

- look up x in the symbol table.
 - If x is there, fetch the level number from the first list item.
 - If that level number = current level then issue a "multiply declared variable" error;
 - otherwise, add a new item to the front of the list with the appropriate type and the current level number.

... **continue**

3. **To process a use of x:** Processa l'uso di x

- **Look up x in the symbol table.**
 - **If it is there, link the use of x with the symbol-table entry at the front of the list**
 - **If it is not there, then issue an "undeclared variable" error.**

4. **On scope exit:** Uscita dallo scope

- **Scan all the names in the symbol table, looking at the first item on each list.**
- **If that item's level number = current level number, then remove it from its list**
 - **and if the list becomes empty, remove the name.**
- **Finally, decrement the current level number.**

Running times

- 1. Scope entry:** Entrare in un nuovo scope richiede solo di incrementare il contatore del livello di annidamento (nesting level).
 - time to increment the level number, $O(1)$.
- 2. Process a declaration:** L'inserimento di un nuovo nome richiede di Trovare l'identificatore nella Hash Table, Controllare la testa della lista per verificare l'ombreggiamento/duplicazione (dipendente dallo scope corrente) ed Aggiungere la nuova entry in testa alla lista
 - using hashing, constant expected time ($O(1)$).
- 3. Process a use:** Si trova il nome nella Hash Table. La dichiarazione corretta e più strettamente annidata è garantita essere in testa alla lista: non è necessario scorrere gli scope.
 - using hashing, constant expected time ($O(1)$).
- 4. Scope exit:**
 - time proportional to the number of names in the symbol table.

Punto Debole

Uscire da uno scope è l'operazione più costosa. L'analizzatore deve Scorrere l'intera Hash Table (tutti gli N nomi) e Per ogni nome, verificare se l'entry in testa alla sua lista appartiene al livello di scope che si sta chiudendo. Se l'entry appartiene a quello scope, l'entry viene rimossa (POP) dalla testa della lista.

TEST YOURSELF

- Assume that the symbol table is implemented using a hashtable of lists.
- Draw pictures to show how the symbol table changes as each declaration in the following code is processed.

```
void g(int x, int a) {  
    double d;  
    while (...) {  
        int d, w;  
        double x, b;  
        if (...) { int a,b,c; }  
    }  
    while (...) { int x,y,z; }  
}
```

Type Checking

Types

Il concetto di Tipo è una delle astrazioni più importanti in informatica e nella teoria dei linguaggi di programmazione. È la base su cui l'analizzatore semantico verifica la coerenza logica e l'uso corretto dei dati.

Nei linguaggi a tipizzazione forte, la nozione di tipo è rigida e la compatibilità è controllata rigorosamente in fase di compilazione.

Nei linguaggi a tipizzazione debole, le conversioni implicite (coercizioni) sono più comuni, rendendo il concetto di tipo più flessibile e talvolta ambiguo.

- What is a type?
 - The notion varies from language to language

Consensus

Nonostante le differenze specifiche, c'è un forte consenso teorico sulla natura di un tipo in un linguaggio formale:

- A set of values
- A set of operations on those values

- Classes are one instantiation of the modern notion of type

I tipi sono un insieme di valori e per ogni tipo viene identificato un insieme di azioni accettate su quei valori

Why Do We Need Type Systems?

Il linguaggio definisce i tipi e le azioni legate ai tipi: dà una interpretazione ai valori

Consider the assembly language fragment

I sistemi di tipi permettono al compilatore (e quindi all'Analisi Semantica) di fare una verifica statica (compile-time check) sul comportamento previsto del programma.

Identificazione di Errori Logici: Molti errori logici comuni, vengono catturati e segnalati come errori semantici ben prima che il codice venga eseguito.

Garanzia di Senso: La regola fondamentale è: se l'operazione sui tipi è valida, l'operazione ha senso nel contesto del linguaggio.

add \$r1, \$r2, \$r3

In Assembly, non esistono i tipi



What are the types of \$r1, \$r2, \$r3?

I tipi sono una forma di documentazione per il codice.

Interfacce Chiare: In un sistema complesso con molte funzioni e moduli, i tipi definiscono in modo esplicito il contratto tra i moduli: quali input sono attesi e quali output sono prodotti.

Manutenzione Semplificata: Se un programmatore modifica una funzione, il sistema di tipi segnala immediatamente tutti gli altri punti del codice che utilizzano quella funzione e che devono essere adattati, riducendo il rischio di errori a cascata.

In sintesi, la disciplina di tipo trasforma un insieme caotico di istruzioni macchina in un sistema organizzato, scalabile e manutenibile, dove l'Analisi Semantica funge da garante di questo ordine.

Types and Operations

- Most operations are legal only for values of some types
 - It doesn't make sense to add a function pointer and an integer in C
 - It does make sense to add two integers
 - But both have the same assembly language implementation!

Type Systems

Specifica la Validità delle Operazioni: Definisce formalmente un insieme di regole di tipo che specificano, per ogni operatore o costrutto del linguaggio, quali tipi sono operandi legali e quale tipo produce il risultato.

- A language's type system specifies which operations are valid for which types

- The goal of type checking is to ensure that operations are used with the correct types

- Enforces intended interpretation of values, because nothing else will!

- Assicurare l'Uso Corretto: L'obiettivo primario è verificare che le operazioni siano usate con i tipi corretti, come stabilito dalle regole del sistema.
- Enforce Intended Interpretation: Il Type Checking è l'unico meccanismo che garantisce l'interpretazione intenzionale dei valori. Il tipo vincola l'hardware a comportarsi come richiesto dall'astrazione del linguaggio.

- Type systems provide a concise formalization of the semantic checking rules

Concisa Formalizzazione: I sistemi di tipi forniscono un modo conciso e matematicamente rigoroso per esprimere le regole di controllo semantico. Regole di Inferenza: Queste regole sono spesso espresse come Regole di Inferenza di Tipo (Type Inference Rules), che stabiliscono le precondizioni sui tipi degli operandi e il tipo che deve risultare dall'espressione.

What Can Types do For Us?

- Can detect certain kinds of errors
- **Memory errors:** i tipi aiutano a prevenire gli usi errati dei puntatori e degli indirizzi.
 - Reading from an invalid pointer, etc.
- **Violation of abstraction boundaries:**

Questo è un concetto fondamentale nell'ingegneria del software, specialmente in linguaggi orientati agli oggetti. I tipi definiscono i contratti e le interfacce, e l'Analisi Semantica si assicura che questi contratti non vengano violati.

```
class FileSystem {  
  open(x : String) : File {  
    ...  
  }  
  ...  
}
```

```
class Client {  
  f(fs : FileSystem) {  
    File fdesc ← fs.open("foo")  
    ...  
  } -- f cannot see inside fdesc !  
}
```

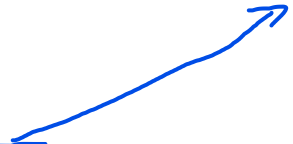
Il sistema di tipi impedisce alla funzione f di accedere o manipolare i dettagli interni (i campi privati) dell'oggetto fdesc che non sono esposti dall'interfaccia pubblica del tipo File.

Type Checking Overview

- Three kinds of languages:

- 1 - *Statically typed*: All or almost all checking of types is done as part of compilation (C) Compile-Time
la maggior parte
- 2 - *Dynamically typed*: Almost all checking of types is done as part of program execution (Scheme) Run-Time
- 3 - *Untyped*: No type checking (machine code)

In Java, parte del type checking viene fatto dalla VM



The Type Wars

- Competing views on static vs. dynamic typing

- Static typing proponents say:

- Static checking catches many programming errors at compile time
- Avoids overhead of runtime type checks

Cattura degli Errori in Fase di Compilazione: L'Analisi Semantica (Type Checking) rileva la maggior parte degli errori di tipo prima che il codice venga eseguito. Ciò porta a software più robusto e affidabile in produzione.

Evitare l'Overhead a Runtime: Una volta che i tipi sono verificati staticamente, il programma in esecuzione non ha bisogno di includere codice per controllare continuamente la validità dei tipi, risultando in un codice binario più veloce e un'esecuzione più efficiente.

- Dynamic typing proponents say:

- Static type systems are restrictive
 - e.g. due to the type statically associated/fixed for a variable
 - for variable "double x" we cannot do "int y=x" even if at run-time "x" actually contains an integer
 - programmers end up using casts escaping the type system
- Rapid prototyping easier in a dynamic type system

Flessibilità e Meno Restrizioni: I sistemi statici sono percepiti come troppo restrittivi. La flessibilità del Dynamic Typing permette di scrivere codice più generale e conciso. Prototipazione Rapida: La mancanza di rigorosi controlli di tipo in fase di compilazione accelera il ciclo di sviluppo, facilitando l'iterazione e la prototipazione rapida. Si può cambiare la struttura del codice molto più facilmente senza dover rifattorizzare le definizioni di tipo ovunque.

Il Type Checking è il processo con cui l'analizzatore semantico garantisce che il codice rispetti le regole stabilite dal sistema di tipi del linguaggio. Il Type Checking tipicamente visita l'AST (spesso in modalità bottom-up) e verifica, nodo per nodo, che i tipi degli operandi siano compatibili con l'operatore.

Relazione con Type Checking: Molti vedono l'inferenza come parte integrante del Type Checking, specialmente quando si tratta di calcolare il tipo di espressioni complesse. Per verificare la validità di un'espressione, si deve prima inferire il tipo di tutte le sue sotto-espressioni.

In sintesi: Type Checking è la verifica (è il codice corretto?), mentre Type Inference è la deduzione (qual è il tipo di questo codice?). In pratica, si fondono in un unico processo che annota l'AST con le informazioni di tipo risolte.

Type Checking and Type Inference

Type Checking is the process of checking that the program ^{obbedisce} obeys the type system

Often involves inferring types for parts of the program

- Some people call the process *type inference*

L'Inferenza di Tipo è l'aspetto del Type Checking che si occupa di dedurre (inferire) il tipo di un'espressione o di una variabile quando questo non è stato dichiarato esplicitamente dal programmatore. Nei linguaggi moderni, non è sempre necessario dichiarare il tipo di una variabile o di una funzione.

Rules of Inference

- We have seen two examples of formal notation specifying parts of a compiler
 - Regular expressions (for the lexer)
 - Context-free grammars (for the parser)
- The appropriate formalism for type checking is logical rules of inference

Le Regole di Inferenza forniscono un metodo matematicamente rigoroso e conciso per specificare quando un'espressione ha un certo tipo, basandosi sui tipi delle sue sotto-espressioni.

Idea: partendo dalle foglie, inferisco i tipi da sottoparti del programma

La struttura delle Regole di Inferenza è ideale perché si adatta perfettamente alla verifica dei costrutti del linguaggio:

- Ipotesi (Premesse): Rappresentano le condizioni che devono essere vere sui tipi delle sotto-espressioni o dei componenti di un costrutto.
- Conclusione: Rappresenta il tipo risultante dell'intera espressione o la validità del costrutto, se le premesse sono soddisfatte.

Why Rules of Inference?

- Inference rules have the form
If Hypothesis is true, then Conclusion is true
- Type checking computes via reasoning
If E_1 and E_2 have certain types, then E_3 has a certain type

Il Type Checker visita l'Enriched AST (spesso in modalità bottom-up) e usa le regole di inferenza per dedurre il tipo di ogni nodo, a partire dai tipi noti dei nodi foglia (costanti e variabili).

- Esempio fornito: Se E_1 (es. la parte sinistra di un'addizione) e E_2 (la parte destra) hanno certi tipi (Ipotesi), allora E_3 (l'intera espressione $E_1 + E_2$) ha un certo tipo (Conclusione).

Questo processo è ricorsivo: il tipo di un nodo padre non può essere determinato finché non è stato determinato il tipo di tutti i suoi nodi figli. Questo ragionamento a ritroso (o deduzione) è formalizzato in modo impeccabile dalle Regole di Inferenza.

From English to an Inference Rule

- The notation is easy to read (with practice)
- Start with a simplified system and gradually add features
- Building blocks
 - Symbol $\wedge$ is “and”
 - Symbol $\Rightarrow$ is “if-then” questo indica l'implica (cioé la finale del costrutto 'se , allora ...')
 - $x:T$ is “x has type T”

From English to an Inference Rule (2)

① If e_1 has type Int and e_2 has type Int,
then $e_1 + e_2$ has type Int

La Regola in Linguaggio Naturale: questa è la definizione umana e intuitiva.

scrittura
equivalente

② (e_1 has type Int $\wedge$ e_2 has type Int) $\Rightarrow$
 $e_1 + e_2$ has type Int

La Regola in Logica Proposizionale: questa è la formalizzazione logica standard.

scrittura
equivalente

③ ($e_1: \text{Int}$ $\wedge$ $e_2: \text{Int}$) $\Rightarrow$ $e_1 + e_2: \text{Int}$

La Regola di Inferenza di Tipo: notazione abbreviata e leggermente ambigua

Proof System (Sistema di Dimostrazione): è la struttura formale e logica che definisce e permette di dimostrare la correttezza di tipo di un programma. Un Proof System è l'insieme formale delle Regole di Inferenza di Tipo del linguaggio. Serve come meccanismo di verifica statica e deduttiva. Il Proof System è l'insieme completo delle regole formali (Regole di Inferenza) che definiscono la semantica dei tipi del linguaggio.

- Scopo: Queste regole coprono ogni costrutto del linguaggio (variabili, costanti, addizione, assegnazione, condizionali, cicli, chiamate di funzione, ecc.).
- Funzione: L'insieme delle regole, nel suo complesso, permette al compilatore (il Type Checker) di fare deduzioni sulla correttezza di tipo di qualsiasi espressione valida nel linguaggio.

From English to an Inference Rule (3)

The statement

$$(e_1: \text{Int} \wedge e_2: \text{Int}) \Rightarrow e_1 + e_2: \text{Int}$$

is a special case of

$$(\text{Hypothesis}_1 \wedge \dots \wedge \text{Hypothesis}_n) \Rightarrow \text{Conclusion}$$

Quest'ultima

The latter is an inference rule usually written:

$$\frac{\vdash \text{Hypothesis}_1 \quad \dots \quad \vdash \text{Hypothesis}_n}{\vdash \text{Conclusion}}$$

Nella teoria dei tipi, il simbolo $\vdash$ (che si legge "deduce" o "giudica") viene utilizzato per indicare un giudizio di tipo valido in un certo contesto.

...and the above rule is written:

$$\textcircled{4} \quad \frac{\vdash e_1 : \text{Int} = \vdash e_2 : \text{Int}}{\vdash e_1 + e_2 : \text{Int}} \quad \text{indica l'implica}$$

È sottinteso che le ipotesi sono separate da AND

Se il "Proof System" è riuscito ad inferire i tipi dell'ipotesi, allora anche la Conclusione è vera

Notation for Inference Rules

- By tradition inference rules are written

$$\frac{\vdash \text{Hypothesis}_1 \quad \dots \quad \vdash \text{Hypothesis}_n}{\vdash \text{Conclusion}}$$

Nella Regola di Inferenza, è sottointeso che la regola vale per ogni variabile usata

- Type rules have hypotheses and conclusions of the form:

$$\vdash e : T$$

è un'asserzione (o giudizio) che afferma che è possibile dedurre che l'espressione o variabile x ha tipo T , seguendo le regole formali del Sistema di Dimostrazione (Proof System) del linguaggio.

- $\vdash$ means “we can prove that . . .”

Quando il Type Checker attraversa l'AST, sta costruendo un albero di prova dimostrando la validità di questi giudizi, culminando in un giudizio finale per l'intero programma.

An example

- An inference system for proving $x < y$

$$\frac{}{\vdash x < (x+1)}$$

[A]  Regola A
Una regola é un'assioma se non ha ipotesi sopra l'implica (cioè sopra la barra)
(Axiom stating that every number is strictly smaller than its successor)

$$\frac{\vdash x < y \quad \vdash y < z}{\vdash x < z}$$

[T]  Regola T
(Transitivity of $<$)

An example (continued)

Utilizzando il Proof System, costruiamo l'Albero di Inferenza, dove per ogni nodo (pezzo del programma), applico una delle regole (per applicarla, scelgo un numero, perché il Proof System vale per ogni valore delle variabili)

- How to prove that $6 < 10$?

$\vdash 6 < 10$

Non posso applicare l'Assioma [A] perché 6 e 10 sono numeri distanti, non adiacenti, quindi applico la Regola [T]

An example (continued)

- How to prove that $6 < 10$?

$\vdash 6 < 7$

↗ Applico l'Assioma [A]

$\vdash 7 < 10$

↗

Applico la Regola [T]

$\vdash 6 < 10$

[T]

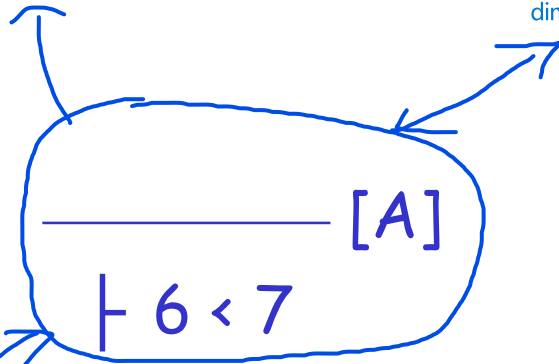
Ogni ipotesi deve
essere verificata,
affinché la
conclusione finale
sia verificata

An example (continued)

- How to prove that $6 < 10$?

Applicando l'Assioma, essendo che non ha ipotesi da verificare, ho completato la verifica di quella sottoparte di programma

Gli assiomi sono le Regole di Inferenza che non hanno premesse (o le cui premesse sono sempre vere). Non richiedono una dimostrazione ricorsiva; sono fatti fondamentali.



$\vdash 7 < 10$

[T]

$\vdash 6 < 10$

Applicare un assioma ad una parte del programma significa che il Type Checker ha verificato quella parte come semanticamente valida sulla base delle definizioni fondamentali del linguaggio, e quel risultato (il tipo dell'espressione) può essere usato come premessa per deduzioni successive (altre regole di inferenza).

An example (continued)

- How to prove that $6 < 10$?

Continuo ad applicare le Regole T e A

$$\begin{array}{c} \frac{}{\vdash 6 < 7} [A] \quad \frac{\vdash 7 < 9 \quad \vdash 9 < 10}{\vdash 7 < 10} [T] \\ \hline \vdash 6 < 10 \end{array}$$

An example (continued)

- How to prove that $6 < 10$?

$\vdash 6 < 7$	$\vdash 7 < 9$	$\vdash 9 < 10$	[A]
[A]			[T]
$\vdash 7 < 10$			
$\vdash 6 < 10$			[T]

An example (continued)

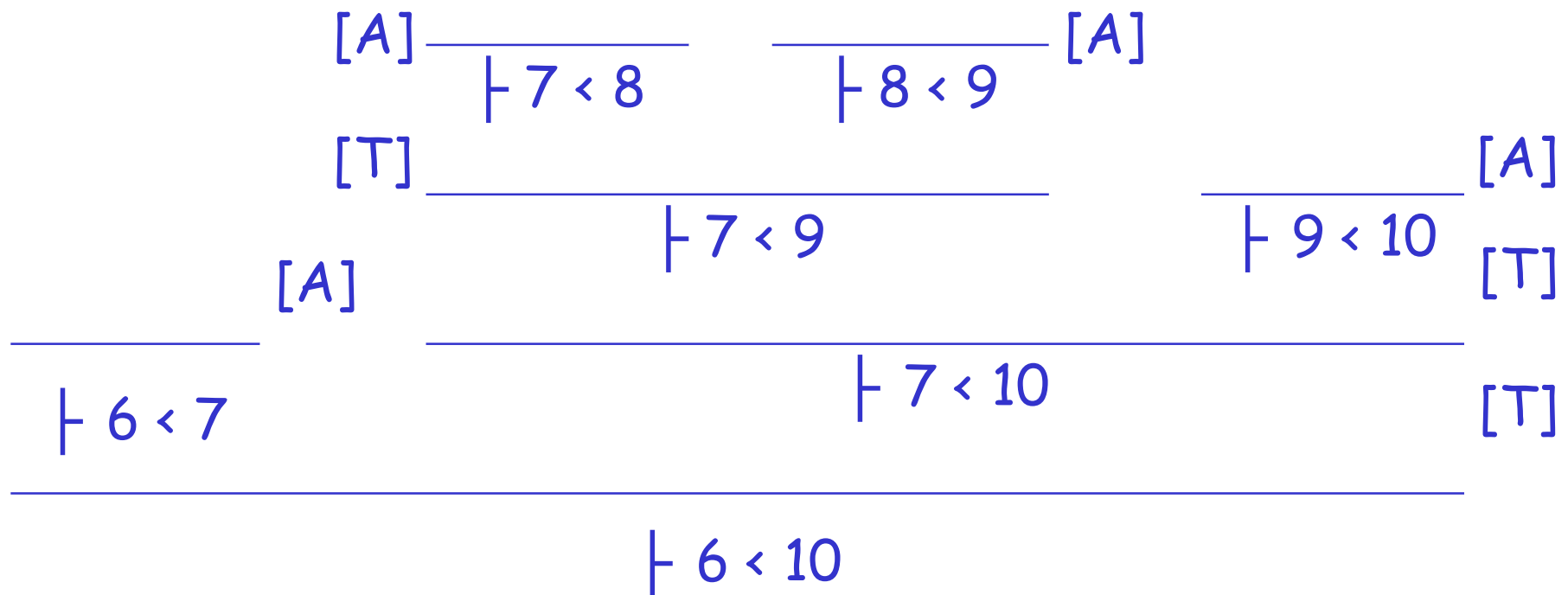
- How to prove that $6 < 10$?

$$\begin{array}{c} \begin{array}{ccc} & \vdash 7 < 8 & \vdash 8 < 9 \\ [T] & \hline & \vdash 7 < 9 & \\ [A] & & \end{array} & \begin{array}{c} \hline \vdash 9 < 10 \\ [A] \\ \hline \vdash 9 < 10 \\ [T] \end{array} \\ \hline \begin{array}{ccc} \vdash 6 < 7 & & \vdash 7 < 10 \\ [A] & & [T] \end{array} \\ \hline \vdash 6 < 10 \end{array}$$

An example (continued)

Fino a costruire un albero con radice in basso (questo albero definisce come vere tutte le sottoparti del programma e quindi la conclusione iniziale é vera)

- How to prove that $6 < 10$?



Esempio Per l'espressione $(a + 5) * b$:
- Fase 1 (Name Resolution): I nodi a e b vengono annotati con puntatori alle loro dichiarazioni nella Tabella dei Simboli, rivelando i loro tipi (es. a: Int, b: Int).
- Fase 2 (Type Checking):
> Sottoespressione $a + 5$: Il Type Checker legge $\text{tipo}(a) = \text{Int}$ e $\text{tipo}(5) = \text{Int}$. Applica la regola [Add]. Il nodo $a + 5$ viene annotato con il tipo Int.
> Espressione $(a + 5) * b$: Il Type Checker usa il tipo appena annotato Int e $\text{tipo}(b) = \text{Int}$. Applica la regola di moltiplicazione. Il nodo radice $*$ viene annotato con il tipo Int.

Quindi, l'AST arricchito è la struttura completa che contiene sia le informazioni sullo scope (linking) che le informazioni sul tipo (annotazioni) necessarie per le fasi successive del compilatore.

Type system: Two Rules

L'Analisi Semantica si svolge in due fasi principali, e l'AST viene arricchito progressivamente:
- Name Resolution (Fase 1): In questa fase, gli usi delle variabili e delle funzioni vengono collegati (annotati nell'AST) alle rispettive dichiarazioni nella Tabella dei Simboli. Questo risolve il problema dello scoping.
- Type Checking (Fase 2): Dopo che i riferimenti (usi -> dichiarazioni) sono stati risolti, il Type Checker visita l'AST arricchito per: Verificare la compatibilità dei tipi, Annotare in aggiunta il nodo dell'AST con il tipo dedotto per l'espressione o sottoespressione.

Questa è una regola di tipo assiomatico (non ha premesse derivate da altre regole), che stabilisce il tipo base per le costanti: Qualsiasi token che è un intero è verificato per assioma e ha tipo Int.

$$\frac{(i \text{ is an integer token})}{\vdash i : \text{Int}} \quad [\text{Int}]$$

(this kind of premises are usually written as side conditions, as they do not require other inferences)

$$\frac{\frac{\vdash e_1 : \text{Int} \quad \vdash e_2 : \text{Int}}{\vdash e_1 + e_2 : \text{Int}} \quad [\text{Add}]}{\vdash i : \text{Int}} \quad [\text{Int}] \quad (i \text{ is an integer token})$$

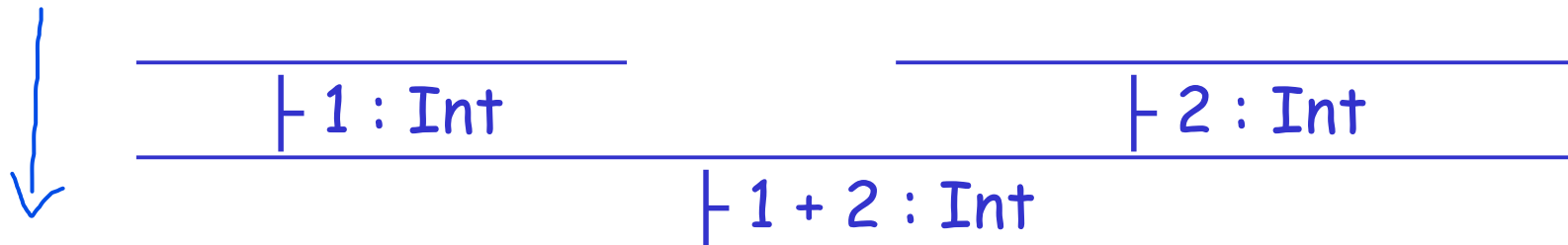
Questa è una regola di deduzione che richiede la verifica ricorsiva dei tipi delle sotto-espressioni. Significato: L'operazione '+' è valida solo se entrambe le sotto-espressioni (e_1 ed e_2) sono dedotte avere tipo Int. Se le premesse sono soddisfatte, l'intera espressione ($e_1 + e_2$) è dedotta avere tipo Int.

L'analizzatore semantico combina i risultati della Name Resolution (ottenuti dalla Tabella dei Simboli) con queste Regole di Inferenza per annotare l'AST con il tipo risolto di ogni espressione.

Type system: Two Rules (Cont.)

Le regole di Inferenza di tipo sono dei template, dove istanzio le regole con valori precisi (dato che la regola vale per ogni valore di quella variabile)

- These rules give templates describing how to type integers and + expressions come tipare
- By filling in the templates, we can produce complete typings for expressions



Applico prima
l'ASSIOMA sui due
int e poi applico la
regola ADD

I sistemi di tipi moderni tendono a privilegiare la soundness a scapito di una completa completeness, preferendo rigettare un po' di codice valido piuttosto che rischiare di produrre codice con errori di tipo nascosti.

Soundness

La soundness è la proprietà che definisce l'affidabilità di un sistema di tipi. In termini semplici, è la garanzia che le conclusioni tratte in fase di compilazione (Analisi Semantica) siano rispettate in fase di esecuzione (run-time).

Soundness (Correttezza):
> Cosa Garantisce: Se il compilatore dice che è giusto, è giusto.
> Problema: Un sistema sound potrebbe rigettare del codice corretto ("troppo restrittivo").
Completeness (Completezza):
> Cosa garantisce: Se il codice è giusto, il compilatore lo accetterà.
> Problema: Un sistema completo potrebbe accettare del codice sbagliato ("non sicuro").

- A type system is sound if
 - Whenever we can infer $\vdash e : T$
 - Then at run-time 'e' actually evaluates to a value of type T

We only want sound rules

La soundness è cruciale per la sicurezza e l'affidabilità del codice:

- Assenza di Errori di Tipo a Runtime: Se un sistema è sound, non si verificheranno "errori di tipo" imprevisti durante l'esecuzione (come tentare di moltiplicare una stringa o accedere a un campo inesistente). Il Type Checker ha già garantito la coerenza.
- Affidabilità del Compilatore: Permette al compilatore di fare assunzioni sicure sui tipi durante la generazione del codice (es. se sa che una variabile è int, può usare istruzioni hardware ottimizzate per gli interi, senza bisogno di check di sicurezza aggiuntivi in fase di esecuzione).
- Realizzazione dell'Astrazione: La soundness garantisce che l'astrazione di alto livello imposta dal linguaggio (i tipi) non sia violata dal comportamento a basso livello della macchina.

In sintesi, la soundness è la proprietà che fa sì che un programma ben tipizzato staticamente sia effettivamente ben tipizzato dinamicamente.

Per ogni tipo di espressione (o costruito sintattico) che appare nell'AST, viene applicata una Regola di Tipo specifica:

> Espressione Aritmetica ($e_1 + e_2$): Viene usata la regola [Add].

> Assegnazione ($x = e$): Viene usata la regola [Assign].

> Condizione (if): Viene usata la regola [If].

Ogni nodo dell'AST (ad eccezione delle foglie) corrisponde a una Regola di Inferenza che deve essere applicata.

Type Checking Proofs

Di solito, una regola per ogni operatore

Associazione regola-espressione

• Type checking proves facts $e : T$

Per ogni tipo di espressione viene utilizzata una regola di tipo.

One type rule is used for each kind of expression,
e.g. previous rule used if e is in the form $e_1 + e_2$

• In the type rule used for typing e :

- The hypotheses are the proofs of types of e 's subexpressions
- The conclusion is the proof of type of e

Ogni regola di tipo formalizza la relazione tra i componenti dell'espressione (sottoespressioni) e l'espressione nel suo insieme:

- Ipotesi (Premesse): Sono le prove (giudizi di tipo) delle sottoespressioni ($e_1, e_2, \dots$).

> Queste premesse devono essere già state dimostrate ricorsivamente nei nodi figli dell'AST (il meccanismo bottom-up).

> Esempio: Nella regola [Add], le ipotesi sono $\vdash e_1 : \text{Int}$ e $\vdash e_2 : \text{Int}$.

- Conclusione: È la prova (giudizio di tipo) dell'intera espressione (e).

> Questa è la nuova asserzione che viene annotata sul nodo corrente dell'AST.

> Esempio: Nella regola [Add], la conclusione è $\vdash e_1 + e_2 : \text{Int}$.

Il Type Checker costruisce un Albero di Prova (Proof Tree) che è la rappresentazione formale della correttezza semantica del programma. Se si riesce a costruire l'albero fino alla radice, il programma è ben tipizzato e, se il sistema è sound, è sicuro.

Rules for Constants

$$\frac{}{\vdash \text{false} : \text{Bool}} \quad [\text{Bool}]$$

false e true sono delle costanti booleane

$$\frac{}{\vdash s : \text{String}} \quad [\text{String}]$$

(s is a string constant)

Object Creation Example

$$\frac{}{\vdash \text{new } C(): C}$$
 Assioma [New] (C denotes a class with
parameterless constructor)

Two More Rules

Premessa: L'espressione interna e deve essere dedotta avere tipo Bool (Booleano).
Conclusione: Se la premessa è soddisfatta, l'intera espressione $\text{not } e$ è dedotta avere tipo Bool.
Errore Semantico: Se e fosse, ad esempio, di tipo Int (es. $\text{not } 5$), il Type Checker non riuscirebbe ad applicare la regola [Not] e segnalerebbe un errore di tipo ("type mismatch").

$$\frac{\vdash e : \text{Bool}}{\vdash \text{not } e : \text{Bool}} \quad [\text{Not}]$$

- Premessa 1 (Condizione e_1): L'espressione condizionale e_1 deve essere dedotta avere tipo Bool.

> Motivazione: È la condizione che decide se il ciclo deve continuare o fermarsi, e le condizioni logiche devono essere booleane.

- Premessa 2 (Corpo del Loop e_2): L'espressione o blocco di codice e_2 deve essere dedotto avere un tipo generico T .

> Nota: La slide usa T generico, ma spesso, se il corpo è un blocco di istruzioni, il suo "tipo" può essere considerato void o Unit, o semplicemente un'istruzione (Stmnt), poiché non produce un valore utilizzabile nell'espressione circostante.

- Conclusione: L'intera costruzione $\text{while } \dots$ è dedotta avere tipo Stmnt (Statement / Istruzione).

> Motivazione: Un ciclo while è un'istruzione che modifica lo stato del programma (effetti collaterali), ma non restituisce un valore che possa essere assegnato ad una variabile.

$$\frac{\begin{array}{c} \vdash e_1 : \text{Bool} \\ \vdash e_2 : T \end{array}}{\vdash \text{while } e_1 \text{ loop } e_2 \text{ pool} : \text{Stmnt}} \quad [\text{Loop}]$$

loop viene usata per '{'
pool viene usata per '}'

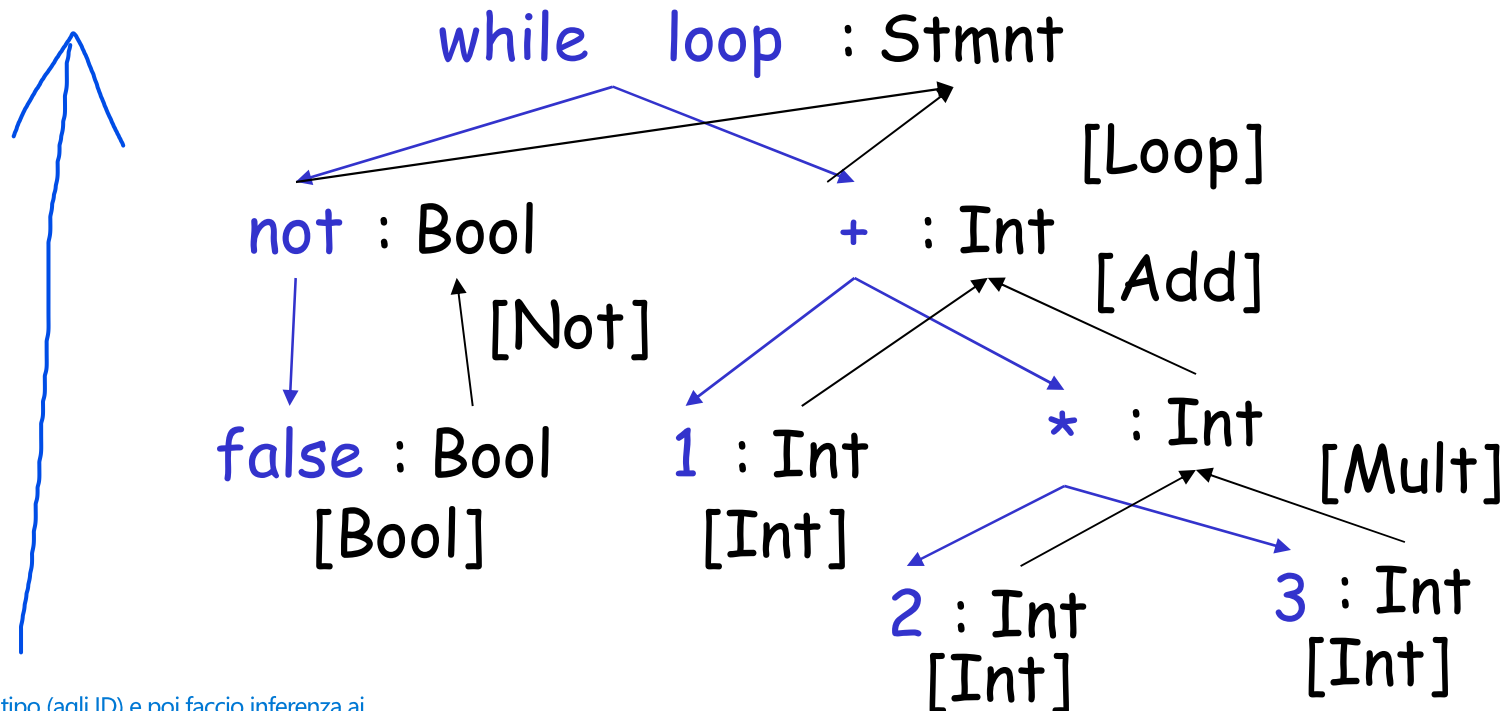
insieme, indicano il
blocco di codice del loop

Queste regole garantiscono che la semantica del costruito while sia rispettata: la condizione è logica, e il costruito stesso è trattato come un'istruzione.

Typing: Example

- Un'espressione è un costrutto che, quando valutato, produce un valore.
 - > Scopo: Calcolare un valore.
 - > Risultato: Ha sempre un tipo dedotto (es. Int, Bool, String) che viene annotato sull'AST durante il Type Checking.
- Un'istruzione è un costrutto che, quando eseguito, compie un'azione (ha effetti collaterali, come modificare lo stato della memoria o controllare il flusso del programma), ma non produce un valore utilizzabile nell'espressione circostante.
 - > Scopo: Eseguire un'azione (controllo del flusso o modifica dello stato).
 - > Risultato: Non ha un tipo standard, o il suo tipo è convenzionalmente indicato come void o Stmtnt (Statement).

- Typing for while not false loop 1 + 2 * 3 pool



Ho un AST, dalle foglie dó il tipo (agli ID) e poi faccio inferenza ai nodi sopra.

Applicare le regole di inferenza, mi servono per tipare il programma, quindi mi serve per verificare la correttezza del programma, oltre a dare i tipi alle sottoparti del programma

Typing Derivations

→ La correttezza semantica del programma viene dimostrata costruendo un albero.

- The typing reasoning can be expressed as a tree:

$$\frac{\frac{\frac{}{\vdash \text{false} : \text{Bool}}}{\vdash \text{not false} : \text{Bool}} \quad \frac{\frac{}{\vdash 1 : \text{Int}} \quad \frac{\frac{\frac{}{\vdash 2 : \text{Int}} \quad \frac{}{\vdash 3 : \text{Int}}}{\vdash 2 * 3 : \text{Int}}}{\vdash 1 + 2 * 3 : \text{Int}}}{\vdash \text{while not false loop } 1 + 2 * 3 \text{ pool} : \text{Stmtnt}}$$

- The root of the tree is the whole expression
- Each node is an instance of a typing rule
- Leaves are the rules with no hypotheses

I giudizi più semplici in cima (senza linee sopra) sono gli assiomi.

A Problem

La regola di inferenza per l'uso di una variabile (identificatore) non può dare un tipo da sola perché, a differenza delle costanti, il tipo di una variabile dipende dal contesto in cui è dichiarata.



What is the type of a variable usage?

Il tipo di un identificatore in uso è il tipo con cui è stato dichiarato.

$$\frac{}{\vdash x : ?}$$

[Id]

(x is an identifier)

This rule does not have enough information to give a type.

– We need an **assumption** of the form “*we are in the scope of a declaration of x with type T*”)

Per risolvere questo problema, la regola di inferenza deve includere un contesto di tipo, formalmente rappresentato da O .
 O è l'astrazione formale della Tabella dei Simboli (TS) in cui i nomi sono mappati ai tipi. In pratica, è l'assunzione di cui abbiamo bisogno: “siamo nello scope di una dichiarazione di x con tipo T ”.

- Definizione: L'elemento O è una funzione che associa gli Identificatori ai loro Tipi.
- Contenuto: Contiene le assunzioni sui tipi delle variabili visibili nello scope corrente.
- Corrispondenza: Questo elemento corrisponde direttamente alle informazioni che sono state elaborate e immagazzinate nella Tabella dei Simboli durante la fase di Name Resolution.

Per tipizzare correttamente un'espressione, specialmente quelle che includono variabili (identificatori), il sistema di inferenza deve avere accesso alle informazioni sullo scope, ovvero alla Tabella dei Simboli.

A Solution: Put more information in the rules!

Let O be a function from Identifiers to Types

The sentence $O \vdash e : T$

is read:

Under the assumption that the variables in the current scope have the types given by O , it is ^{provabile} provable that expression e has type T

- **NOTE:** Corresponds to the information stored in the **symbol table**!

Type Environments

L'Ambiente di Tipo O è l'astrazione formale del contenuto della Tabella dei Simboli.
Scopo: O fornisce i tipi per le variabili libere.
Variabile Libera: Una variabile è considerata libera in una (sotto)espressione se l'occorrenza della variabile si riferisce a una dichiarazione che si trova al di fuori di quell'espressione.

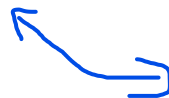
- A type environment O gives types for *free* variables

- A variable is *free* in an (sub)expression if:

- The expression contains an occurrence of the variable that refers to a declaration outside the expression

- E.g. in “int f(int x) {return x+y}” only “y” is free  Mi aspetto che y sia dichiarato in uno scope esterno
- E.g. in the (sub)expression “x”, the variable “x” is free

Il tipo di una variabile bound in uno scope è derivato dalla regola che ha creato quello scope, e non da un contesto esterno. Una volta creato, quel binding è incluso nell'ambiente di tipo per essere utilizzato nel lookup delle sottoespressioni.



Variabile Bound (Legato): L'identificatore è stato dichiarato nello scope corrente o in uno scope immediatamente superiore, e quindi il suo riferimento è risolto internamente a quel costrutto.
Esempio: in 'int f(int x) { return x + 5; }', la variabile x è bound all'interno del corpo della funzione dalla sua dichiarazione come parametro.

Quindi, il type environment O è utile solo per definire il tipo delle variabili free perché quelle bounded sono interne allo scope dove siamo (O contiene tutte le variabili visibili, ma la sua utilità concettuale nell'analisi di una sottoespressione è proprio quella di fornire i tipi per le variabili free che non sono dichiarate lì.)

Modified Rules

Mi porto O nelle regole di inferenza delle espressioni perché posso avere, all'interno di esse, degli usi che hanno dichiarazioni esterne allo scope di quella espressione

The type environment is added to the earlier rules:

$$\frac{}{O \vdash i : \text{Int}} \quad [\text{Int}] \text{ (i is an integer)}$$

$$\frac{\begin{array}{l} O \vdash e_1 : \text{Int} \\ O \vdash e_2 : \text{Int} \end{array}}{O \vdash e_1 + e_2 : \text{Int}} \quad [\text{Add}]$$

New Rules

And we can write new rules:

$$\frac{}{O \vdash x : T}$$

[Id]

(if $O(x) = T$)

Vado a vedere il link che ho con ST
e uso il link, con info di tipo, per
inferire il tipo di quella variabile

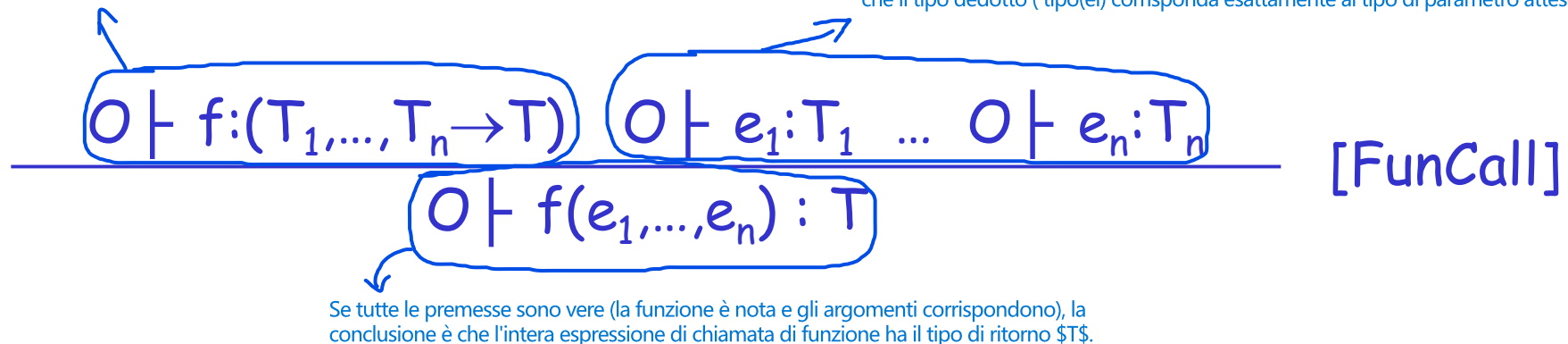
Function invocation

The type of a function is usually written:

- $T_1, \dots, T_n \rightarrow T$
- with $T_1, \dots, T_n$ types of the input parameters
- and T the output return type

Questa premessa verifica che l'identificatore f sia risolto nel contesto O e che il suo tipo sia una funzione che accetta n parametri di tipo $T_1, \dots, T_n$ e restituisce T come output

Queste premesse richiedono che ogni argomento e_i sia ricorsivamente tipizzato e che il tipo dedotto (tipo(e_i)) corrisponda esattamente al tipo di parametro atteso T_i .



Verificare tipo T_1 di e_1 : per capire il suo tipo (es: $x+1$), uso $O[T_0/x]$ (funzione che tipa x come T_0), così ho tipato x e così riesco a tipare e_1 ($x+1$), altrimenti non saprei il tipo di x . Nella conclusione x è bound (perché il let funziona da binder, stabilendo una dichiarazione locale per x e definendone l'ambito di visibilità e il tipo all'interno di e_1), nella premessa x è free.
 - Perché usare $O[T_0/x]$?
 (notazione di replace $[T_0/x]$) Sostituire in O la dichiarazione di x con tipo T_0 (sostituisco quello che c'è già (eventualmente tipo di x già presente))

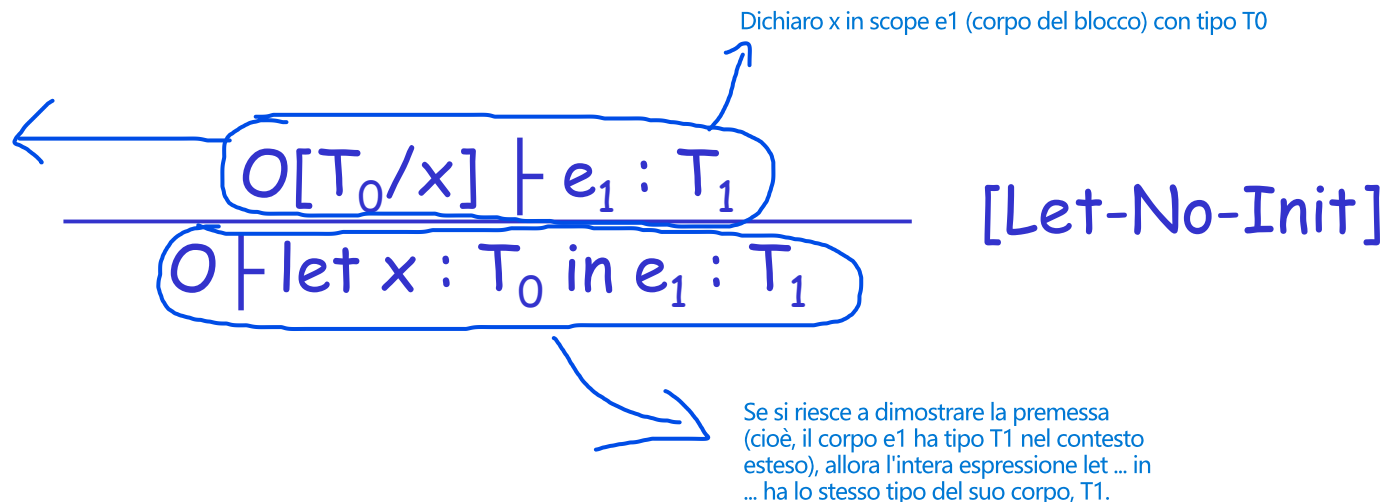
Let (from languages like ML)

- Let statement declares a variable x with given type T_0 that is then defined throughout e_1 :

Il costrutto dichiara una variabile locale x con tipo T_0 , che è valida solo all'interno dell'espressione e_1 .

- $\text{let } x : T_0 \text{ in } e_1$ (without initialization)
- $\text{let } x : T_0 \leftarrow e_0 \text{ in } e_1$ (with initialization)

- La variabile x è legata dal costrutto let.
 - La Notazione $O[T_0/x]$: Questa è la parte cruciale. Significa che il Type Env. O originale viene esteso o modificato per includere il nuovo binding locale: sostituisci l'eventuale binding esistente di x con il nuovo tipo T_0 (o aggiungi il binding $x : T_0$ se non esisteva).
 - Tipizzazione dello Scope Interno (e_1): La premessa richiede di tipizzare l'espressione interna e_1 (il corpo del let) usando il nuovo ambiente $O[T_0/x]$.
 - Questo assicura che qualsiasi uso di x all'interno di e_1 venga risolto correttamente con il tipo T_0 .



Let. Example.

- Consider the expression

let $x : T_0$ in $((\text{let } y : T_1 \text{ in } E_{x,y}) + (\text{let } x : T_2 \text{ in } F_{x,y}))$

(where $E_{x,y}$ and $F_{x,y}$ are some expressions that may contain ^{usi} occurrences of “x” and “y”)

- Declaration
 - of “y” applies to $E_{x,y}$
 - of outer “x” applies to $E_{x,y}$
 - of inner “x” applies to $F_{x,y}$
- This is captured precisely in the typing rule.

L'arricchimento dell'AST avviene nella prima fase dell'analisi semantica in modo top-down. Poi la parte di type checking e definizione dei tipi delle sottoparti del programma avviene nella seconda fase dell'analisi semantica in modo bottom-up

Let Example.

Poiché entrambi i rami sono dimostrati avere tipo int, la regola [Add] può essere applicata, e l'intera espressione ha tipo int. Questa derivazione dimostra il processo bottom-up in cui: Gli assiomi (tipi di funzione noti come len, tipi costanti) sono le foglie. Le regole [Let] e [FunCall] modificano e verificano il contesto man mano che si risale l'albero. Il tipo dell'intera espressione è dedotto solo dopo aver dimostrato i tipi di tutte le sottoespressioni.

AST
Type env.

Types

Scendendo l'albero mi porto dietro le dichiarazioni di variabili free

Aggiunto x con tipo int al Type Env.

$O \vdash \text{let } x : \text{int in} \quad : \text{int} \quad (O(\text{len}) = \text{Str} \rightarrow \text{int})$

$O[\text{int}/x] \vdash \quad + \quad : \text{int}$

$O[\text{int}/x] \vdash \text{let } y : \text{Str in} \quad : \text{int}$

$O[\text{int}/x] \vdash \text{let } x : \text{Str in} \quad : \text{int}$

Aggiunto x con tipo string al Type Env, che nasconde la precedente dich.

$(O[\text{int}/x])[\text{Str}/x] \vdash \text{len}() \quad : \text{int}$

$(O[\text{int}/x])[\text{Str}/y] \vdash \quad x \quad : \text{int}$

$(O[\text{int}/x])[\text{Str}/x] \vdash \quad x \quad : \text{Str}$

Aggiunto y con tipo string al Type Env.

L'analisi semantica non avviene in un'unica direzione, ma richiede un passaggio bidirezionale sull'Albero Sintattico Astratto (AST) per completare la verifica dei tipi.

1. Fase Top-Down: Propagazione dell'Ambiente di Tipo: Questa fase corrisponde al passaggio dall'alto verso il basso, dalla radice alle foglie, e gestisce l'informazione contestuale. Il Type Environment O è l'astrazione della Tabella dei Simboli e fornisce i tipi per gli identificatori free (variabili dichiarate al di fuori dello scope corrente). L'Ambiente O viene passato lungo l'AST dalla radice verso le foglie. Questo assicura che, quando si raggiunge una sottoespressione, si abbia sempre accesso ai tipi delle variabili dichiarate negli scope esterni. Arricchimento AST (Name Resolution): Nelle implementazioni di compilatori, questo passaggio "verso il basso" non solo propaga l'ambiente, ma genera anche una versione arricchita dell'AST. In questa fase, gli identificatori usati vengono collegati (tramite un puntatore) alle loro entry nella Tabella dei Simboli (che include il loro tipo).

2. Fase Bottom-Up: Calcolo dei Tipi: Questa fase corrisponde al passaggio dal basso verso l'alto, ed è il cuore dell'inferenza di tipo. I tipi vengono calcolati risalendo l'AST, dalle foglie verso la radice. Per ogni nodo (espressione), il Type Checker applica la Regola di Inferenza di Tipo specifica. Le foglie (costanti, variabili) ottengono il loro tipo da assiomi o dal lookup nell'ambiente arricchito (fase 1). I nodi interni usano i tipi calcolati dai figli come premesse per dedurre e annotare il tipo del nodo padre. Questo processo garantisce che la deduzione del tipo (Bottom-Up) abbia accesso alle informazioni contestuali necessarie (dalla fase Top-Down).

Notes

- The type environment ^{assegna} gives types to the free identifiers in the current scope
- The type environment is passed down the AST from the root towards the leaves (Top-Down - Fase 1)
- Types are computed up the AST from the leaves towards the root (Bottom-Up - Fase 2)
- ^{Top-Down} NOTE: in compiler implementations, the "downward" phase generates an enriched version of the AST where the identifiers are linked to their symbol table entry (indicating their type)

Let with Initialization

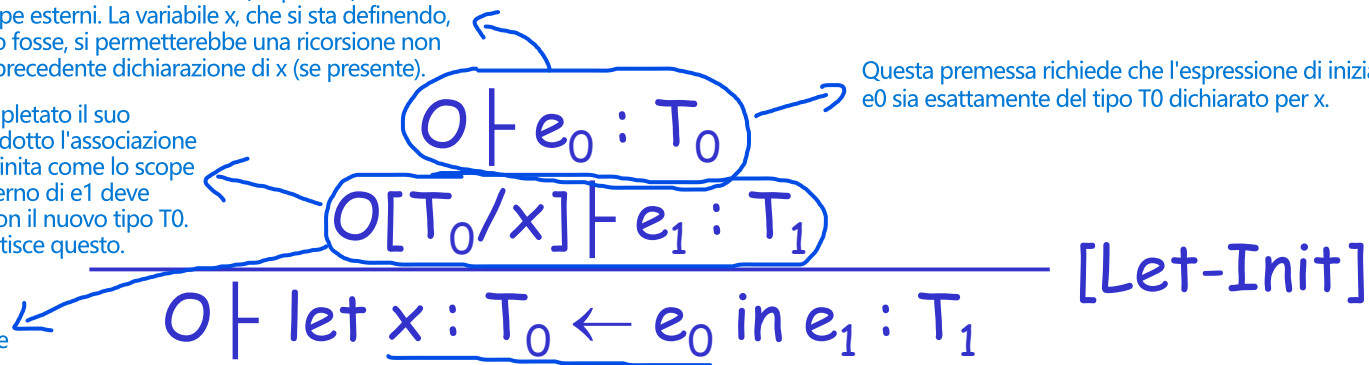
Consider a let with initialization:

L'espressione di inizializzazione e_0 deve essere valutata (e tipizzata) in base alle variabili già dichiarate negli scope esterni. La variabile x , che si sta definendo, non è ancora visibile in e_0 . Se lo fosse, si permetterebbe una ricorsione non banale, o x si riferirebbe a una precedente dichiarazione di x (se presente).

Il costrutto `let` ha ormai completato il suo compito di binder e ha introdotto l'associazione $x : T_0$. L'espressione e_1 è definita come lo scope di x . Qualsiasi uso di x all'interno di e_1 deve quindi poter essere risolto con il nuovo tipo T_0 . La notazione $O[T_0/x]$ garantisce questo.

Questa premessa richiede che l'espressione di inizializzazione e_0 sia esattamente del tipo T_0 dichiarato per x .

La notazione $O[T_0/x]$ è presente solo nella seconda premessa perché è lì che inizia la visibilità (lo scope) della nuova variabile x .



This rule is too “restrictive”. Why?

Let with Initialization

- Consider the example:

class C inherits P { ... }

...

let (x : P) ← new C in ...

...

- The previous let rule does not allow this code
 - We say that the rule is too “restrictive”

Subtyping

- Define a relation $X \leq Y$ on classes to say that:

- An object of type X can be used when one of type Y is acceptable, or equivalently
- X è conforme a conforms with Y

- Also known as “ X is a subclass of Y ”

Se un tipo X è un sottotipo di Y , allora un oggetto di tipo X può essere usato ovunque sia accettabile un oggetto di tipo Y .

- Consider a relation $\leq$ on classes

$$X \leq X$$

$$X \leq Y \text{ if } X \text{ inherits from } Y$$

Ereditarietà diretta

$$X \leq Z \text{ if } X \leq Y \text{ and } Y \leq Z$$

Catena di Ereditarietà

Let with Initialization (Again)

significa che il tipo dedotto T deve essere un sottotipo o un tipo compatibile con il tipo dichiarato T_0 .

$$\frac{\begin{array}{c} O \vdash e_0 : T \\ T \leq T_0 \end{array} \quad O[T_0/x] \vdash e_1 : T_1}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1} \text{ [Let-Init]}$$

Tipo generico che sia $\leq$ a T_0

- Both rules for let are sound
- But more programs type check with the latter, which is more “flexible”

con l'ultima regola che è più flessibile

Entrambe le versioni della regola let (quella rigida e quella flessibile) sono sound. Entrambe garantiscono che, se il controllo dei tipi ha successo, il programma non avrà errori di tipo a run-time. La nuova regola è preferita perché più programmi corretti superano il type checking.

Let with Subtyping. Notes.

trade-off
• There is a tension between

- “Flexible” rules that do not constrain programming
- “Restrictive” rules that ensure safety of execution

La progettazione di un sistema di tipi, come si è visto nell'evoluzione della regola [Let-Init], è sempre un equilibrio tra due obiettivi contrastanti:

1. La Sicurezza (Regole Restrittive): Le regole restrittive sono quelle che impongono condizioni rigide, come la stretta uguaglianza dei tipi.

> Vantaggio: Garantiscono la Soundness (Correttezza) del sistema di tipi. Un programma che supera un controllo rigoroso è estremamente improbabile che incorra in errori di tipo a run-time (esecutivi).

> Svantaggio (Non Completeness): Rigettano del codice che in realtà sarebbe semanticamente corretto (ad esempio, rigettare l'assegnazione di un sottotipo), rendendo il sistema incompleto (non accetta tutto il codice valido).

2. La Praticità (Regole Flessibili): Le regole flessibili sono quelle che introducono meccanismi per accettare un range più ampio di tipi, come l'uso della relazione di sottotipaggio.

> Vantaggio: Riducono le restrizioni sulla programmazione e permettono a più programmi di superare il controllo dei tipi (sono più espressive e facili da usare). Permettono concetti potenti come il polimorfismo (un oggetto può avere tipi multipli).

> Svantaggio (Rischio): Se le regole di flessibilità non sono definite con estrema cura, esiste il rischio di introdurre bug nel sistema di tipi, compromettendo la Soundness (esempio, array covarianti in Java).

Expressiveness of Static Type Systems

- A static type system enables a compiler to detect many common programming errors
- The cost is that some correct programs are disallowed
 - Some argue for dynamic type checking instead
 - Others argue for more expressive static type checking
- But more expressive type systems are also more complex

Il tipo statico è il tipo utilizzato dal compilatore per effettuare il Type Checking e garantire la correttezza del programma. È la classe *C* utilizzata per dichiarare la variabile. È una nozione di compile-time (tempo di compilazione). Questa nozione si estende anche alle espressioni che coinvolgono variabili oggetto. Il Type Checker utilizza il tipo statico per verificare che tutte le chiamate a metodi e gli accessi ai campi siano validi. Ad esempio, se il tipo statico è Animale, non si può chiamare un metodo specifico di Cane perché Animale non lo garantisce.

Static and Dynamic Types

- The *static type* of an object variable is the class *C* used to declare the variable.
 - This notion is then extended to expressions over object variables.
 - A compile-time notion
- The *dynamic type* of an object variable is the class *C* that is used to create its object value (“*new C*”).
 - This notion is then extended to expressions over object variables.
 - A run-time notion

Il tipo dinamico è il tipo effettivo dell'oggetto che la variabile sta referenziando nella memoria in un dato momento. È la classe *C* che è stata usata per creare il valore dell'oggetto (tramite *new C*). È una nozione di run-time (tempo di esecuzione). Anche questa nozione si estende alle espressioni che coinvolgono variabili oggetto. Il tipo dinamico è essenziale per implementare il late binding, dove il metodo corretto da eseguire viene scelto a run-time in base al tipo reale dell'oggetto, e non solo al tipo statico della variabile.

Static and Dynamic Types. (Cont.)

- ^{Nei vecchi} In early type systems, static types correspond directly with dynamic types

Non ci sono variazioni del tipo effettivo a run-time rispetto a quello dichiarato a compile-time.

- Soundness theorem: for all expressions E

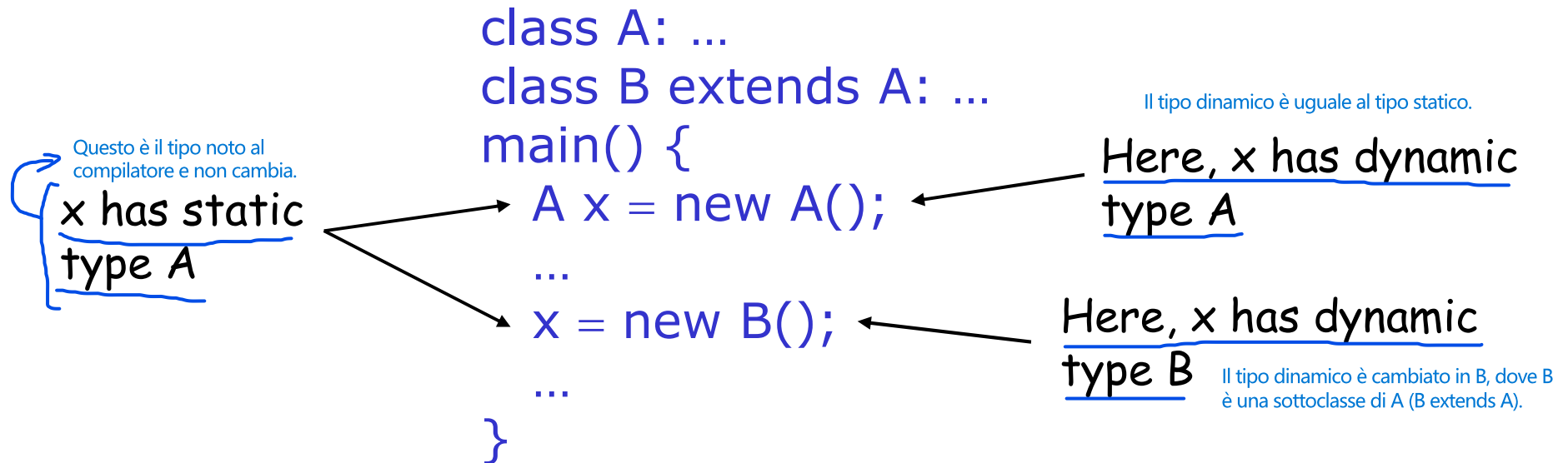
$$\text{dynamic_type}(E) = \text{static_type}(E)$$

(in all executions, E ^{viene valutato in base ai} evaluates to values of the type ^{dedotto dal} inferred by the compiler ^{quindi, il tipo statico})

- This gets more complicated in ^{modern} advanced type systems

Se il Type Checker (che lavora con i tipi statici) accetta il programma, allora l'esecuzione (che gestisce i tipi dinamici) sarà type-safe e non incorrerà in errori di tipo (come tentare di chiamare un metodo inesistente su un oggetto).

Ex. of Code that Type Checks with the Rules



- According to rules: variable x with static type A can be assigned values of type B if $B \leq A$

- **Liskov substitution principle:** subtypes can be used in place of supertypes

I sottotipi possono essere usati al posto dei loro supertipi.

Quindi

- Hence: $\text{dynamic_type}(x) \leq \text{static_type}(x)$

Significato: Il tipo reale dell'oggetto in memoria sarà sempre un sottotipo (o uguale) al tipo dichiarato per la variabile.
Sicurezza: Poiché il compilatore verifica che vengano chiamati solo i metodi definiti nella classe A (il tipo statico), e poiché i sottotipi B garantiscono l'implementazione di tutti i metodi di A (per l'Liskov Sub. Principle), l'esecuzione rimane type-safe.

Soundness of the Type Checking System

Soundness theorem:

$$\forall E. \text{dynamic_type}(E) \leq \text{static_type}(E)$$

Why is this Ok?

- For E , compiler uses $\text{static_type}(E)$ (call it C)
- All operations that can be used on an object of type C can also be used on an object of type $C' \leq C$
 - Objects of type C' must expose all the public methods and public fields exposed by C
 - Objects of type C' could have additional public methods and fields, but they will be not used if the static type is C

Perché il principio di Liskov va bene?

- A compile-time, il compilatore fa type checking su static type

- A run-time, usando i sottotipi, essi dovranno avere i metodi pubblici di C esposti, ma non si potranno usare metodi aggiuntivi (non presenti in C).

Let. Examples.

- Consider the following class definitions

Class A { a() : Int { return 0 }; }

Class B inherits A { b() : Int { return 1 }; }

- An instance of B has methods “a” and “b”
- An instance of A has method “a”
 - A run-time error occurs if we try to invoke method “b” on an instance of A
- Supertype cannot be used in place of a subtype!

Example of Wrong Let Rule (1)

- Now consider a hypothetical let rule:

$$\frac{O \vdash e_0 : T \quad T \leq T_0 \quad O \vdash e_1 : T_1}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1}$$

Manca la dichiarazione nel type env (da aggiungere così: $O[T_0/x] \vdash e_1 : T_1$). È un errore perché se uso x in e_1 non ho la dichiarazione in O e quindi non so il tipo di x più vicino allo scope e_1

- How is it different from the correct rule?

→ Alcuni programmi buoni non passano il type checking

- The following good program does not typecheck

$\text{let } x : \text{Int} \leftarrow 0 \text{ in } x + 1$

→ Alcuni programmi cattivi passano il type checking

- And some bad programs do typecheck

$\text{foo}(x : B) : \text{Int} \{ \text{let } x : A \leftarrow \text{new } A() \text{ in } x.b() \}$

Example of Wrong Let Rule (2)

- Now consider another hypothetical let rule:

Questa regola è sbagliata perché e_0 viene dichiarato di tipo T , ma se do ad x , di tipo T_0 , un'assegnazione di tipo $T \geq T_0$, sto invertendo la regola di sottotipaggio (in questo caso, sopratipo assegnato a sottotipo)

$$O \vdash e_0 : T$$

$$T_0 \leq T$$

$$O[T_0/x] \vdash e_1 : T_1$$

$$O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1$$

- How is it different from the correct rule?

Viene considerato buono, anche se non lo è perché assegno sopratipo a sottotipo.

- The following bad program is well typed

$\text{let } x : B \leftarrow \text{new } A() \text{ in } x.b()$

- Why is this program bad?

inverte la direzione della relazione di sottotipaggio ($\leq$) richiesta, compromettendo la sicurezza del sistema di tipi (Soundness).

Recall the correct Let-Init rule

$$\frac{\begin{array}{c} O \vdash e_0 : T \\ T \leq T_0 \\ O[T_0/x] \vdash e_1 : T_1 \end{array}}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1} \quad [\text{Let-Init}]$$

- We also need a rule for just assigning a value to an already declared variable x

Assignment of an already declared x

x già dichiarato e quindi già presente in Type Environment

$$\frac{\begin{array}{c} O \vdash e_0 : T \\ T \leq T_0 \\ O \vdash e_1 : T_1 \end{array}}{O \vdash x \leftarrow e_0 ; e_1 : T_1} \quad [\text{Assign}] \quad (\text{if } O(x) = T_0)$$

- E.g. the following program is well typed:

let $x : A \leftarrow \text{new } B()$ in $\{ \dots x \leftarrow \text{new } A(); x.a() \}$

Sottotipo assegnato a sopratipo

Il programma è ben tipizzato perché, anche se il tipo dinamico dell'oggetto referenziato da x cambia da B ad A , in ogni momento, il tipo dinamico è sempre un sottotipo (o uguale) del tipo statico A , rispettando la Soundness.

Example of Wrong Let Rule (3)

- Now consider another hypothetical let rule:

$$\frac{O \vdash e_0 : T \quad T \leq T_0 \quad O[T/\underline{x}] \vdash e_1 : T_1}{O \vdash \text{let } x : T_0 \leftarrow e_0 \text{ in } e_1 : T_1}$$

Questa modifica è un errore perché lo scopo di $\text{let } x : T_0$ è stabilire il tipo statico di x come T_0 . Questo tipo statico deve essere utilizzato per la verifica di tipo per tutto lo scope di x (e_1).

cioè viene assegnato ad x il tipo dinamico invece del tipo statico e questo porta a non permettere di ri-assegnare ad x il sopratipo

- How is it different from the correct rule?
- The following good program is not well typed
 $\text{let } x : A \leftarrow \text{new } B() \text{ in } \{ \dots x \leftarrow \text{new } A(); x.a() \}$
- Why is this program not well typed?

Quando il Type Checker entra in e_1 , l'ambiente gli dice che il tipo statico di x è B ($O[B/x]$).

Poiché il supertipo A non è un sottotipo del sottotipo B , l'assegnazione fallisce il Type Check.

Function invocation with subtyping

Function f with type:

- $T_{0,1}, \dots, T_{0,n} \rightarrow T$
- with $T_{0,1}, \dots, T_{0,n}$ types of the input parameters
- and T the output return type

f con output T

La funzione accetta come parametri tipi che sono minori o uguali al tipo $T_{0,i}$

Premette che $e_1 \dots e_n$ sono dei tipi minori o uguali a $T_{0,i}$

$$\frac{O \vdash f : (T_{0,1}, \dots, T_{0,n} \rightarrow T) \quad O \vdash e_1 : T_1 \dots O \vdash e_n : T_n \quad \forall i \ T_i \leq T_{0,i}}{O \vdash f(e_1, \dots, e_n) : T} \text{ [Fun]}$$

Se le premesse sono vere, allora f è di tipo T

A (generally) wrong subtyping rule

- Let array of A be the type of an array containing data of type A
- Intuitively, one could assume:
 - If $B \leq A$ then array of $B \leq$ array of A
(known as "covariant" arrays, present e.g. in Java)
- But, consider the following program (well typed according to this assumption):

```
let function f(x:array of A) {x[1]←new A}  
in let z:array of B  
    in {f(z); z[1].b();};
```
- What is wrong with this program?
 - see next slide...

Il problema è che si è riusciti a inserire un supertipo (A) in un array destinato a contenere sottotipi (B). In questo modo, il supertipo è stato usato al posto del sottotipo, violando la sicurezza.

A (generally) wrong subtyping rule (cont.)

- Problem: A compile-time non dá errore, ma a run-time dá errore (non é Type Safe)

- When the array of subtypes is used in place of an array of supertypes...
- ...it is possible to insert a supertype in the array...
- ...and then the supertype can be used in place of a subtype (type error!)

quindi é immutabile

But if arrays cannot be written/modified, "covariance" is sound!

- It is only possible to use the content of array cells, that is of subtype in place of supertype

La covarianza è sound solo se è possibile solo usare il contenuto delle celle dell'array, perché in quel caso la direzione del sottotipaggio è garantita: `dynamic_type <= static_type`.

Class subtyping

- Subclasses can usually **override some declarations** of the superclass
 - Usually the body of methods
- Assume it is possible to **override both fields and methods**, by changing also their types
 - This will be possible in our FOOL language

La regola generale del sottotipaggio richiederebbe che $B \leq A$ se e solo se T' ha una relazione di sottotipaggio con T .

Il Campo come Array: Un campo può essere visto come una cella di un array. Se ammettiamo l'overriding con sottotipaggio, si verifica il problema di sicurezza discusso per gli array covarianti:

> Sottotipaggio (Covariante) Viziato: Si stabilisce che $B \leq A$ e si applica il sottotipaggio del campo: $T' \leq T$.

> Assegnazione: Si assegna un'istanza di B a una variabile dichiarata come A : $A x = \text{new } B()$.

> Scrittura (Set): Poiché il tipo statico di x è A , il Type Checker permette di scrivere nel campo $x.f$ un valore di tipo T .

> Errore a Run-time: L'oggetto effettivo ($\text{new } B()$) ora contiene un valore di tipo T (il supertipo), ma la classe B si aspetta solo valori di tipo T' (il sottotipo). Se un altro pezzo di codice tenta di accedere a f aspettandosi un T' , si ha un errore di tipo dinamico.

Field overriding

Il problema centrale con l'overriding dei campi (field overriding) è che un campo in un oggetto non è solo una variabile, ma una cella di memoria che può essere modificata dinamicamente.

L'Inserimento Viziato

(Scrittura): Poiché il compilatore sa che il tipo statico di x è A , permette di scrivere nel campo $x.f$ qualsiasi valore che sia compatibile con il tipo T :
 $x.f =$

`nuovo_valore_di_tipo_T;`

Problema: A questo punto, l'oggetto in memoria è ancora un'istanza di B (il tipo dinamico), ma il suo campo f contiene un valore di tipo T (il supertipo). Il tipo T è troppo generico per la classe B . L'istanza di B si aspetta che f contenga sempre valori di tipo T' (il sottotipo).

- Let's start by overriding fields

- Class $A\{\dots, T:f, \dots\}$

- Class B inherits $A\{\dots, T':f, \dots\}$

- Fields can be seen as array cells

- So if they can be dynamically modified, their type T cannot be changed in the subclass

→ IN GENERAL FIELD SUBTYPING IS NOT SOUND
(for this reason in Java we cannot override fields)

- But if they are **immutable** (the fields cannot be modified in FOOL) subtyping is admitted:

- if $T' \leq T$ (for all fields) then $B \leq A$

we call these "covariant fields"

Successivamente, un altro pezzo di codice (o un metodo interno a B) accede al campo f tramite il tipo dinamico B , aspettandosi un valore di tipo T' : $T' \text{ val} = x.f$; // Il codice si aspetta T'
Risultato: Si estrae un valore di tipo T (il supertipo).
Fallimento: Questo valore di tipo T può non avere le proprietà o i metodi attesi dal codice che lavora con B , causando un errore di tipo a run-time.

La sicurezza del sottotipaggio di classe $B \leq A$ è mantenuta se, per ogni metodo m sovrascritto in B (il sottotipo), le seguenti due condizioni sono soddisfatte:

- Covarianza del Tipo di Ritorno: Quando un metodo m viene chiamato su una variabile di tipo statico A , il codice si aspetta che il risultato sia del tipo T (il tipo di ritorno in A). Se il tipo dinamico è B , il metodo restituirà T' . Il tipo di ritorno nel sottotipo (T') deve essere un sottotipo del tipo di ritorno nel supertipo (T). Motivazione (Sicurezza in Lettura): Poiché, per il Principio di Sostituzione di Liskov (LSP), il valore restituito di tipo T' può essere usato in modo sicuro ovunque il codice si aspetti un valore di tipo T .
- Controvarianza dei Tipi dei Parametri: Quando un metodo m viene chiamato su una variabile di tipo statico A , il codice fornisce parametri di tipo T_i (i tipi richiesti da A). La versione in B (il tipo dinamico) deve poter accettare questi parametri in modo sicuro. Il tipo del parametro nel supertipo (T_i) deve essere un sottotipo del tipo del parametro nel sottotipo (T'_i). Motivazione (Sicurezza in Scrittura): Se il metodo in B si aspetta T'_i , e il chiamante fornisce un T_i , l'LSP è soddisfatto: l'argomento T_i (il sottotipo) può essere usato al posto del parametro atteso T'_i (il supertipo).

Method overriding

- Consider now the possibility to override methods by changing the return and, also, the parameter types (not possible in Java):
 - Class $A\{\dots, T:m(T_1:p_1, \dots, T_n:p_n) \{e\}, \dots\}$
 - Class B inherits $A\{\dots, T':m(T'_1:p'_1, \dots, T'_n:p'_n) \{e'\}, \dots\}$
- Consider "let $x:T \leftarrow y.m(e_1, \dots, e_n)$ in e " assuming " y " with static type A and dynamic type B
 - The B return type T' must be usable in place of the A return type T
 - we need $T' \leq T$
 - A parameter types must be usable in place of B parameter types
 - we need $T_i \leq T'_i$

Method overriding (cont.)

- Summarising:
 - if $T_1 \leq T'_1 \dots T_n \leq T'_n$ and $T' \leq T$ (for all methods)
then $B \leq A$
 - The type of m in A is: $T_1, \dots, T_n \rightarrow T$
 - The type of m in B is: $T'_1, \dots, T'_n \rightarrow T'$
 - The general subtyping rule for functions is:

$$\frac{T_1 \leq T'_1 \dots T_n \leq T'_n \text{ and } T' \leq T}{(T'_1, \dots, T'_n \rightarrow T') \leq (T_1, \dots, T_n \rightarrow T)}$$

"covariant" output (return) type

"contravariant" input (parameter) types

Qualsiasi modifica, anche apparentemente innocua, a una regola di tipizzazione ha conseguenze drastiche, muovendosi sempre lungo due assi opposti, come si è visto con le varie regole let:

- Rischio di Unsoundness (Non-Correttezza): Se una regola è resa troppo flessibile, può rendere il sistema di tipi unsound.
- > Conseguenza: I programmi cattivi (quelli che causerebbero errori di tipo a run-time) vengono erroneamente accettati come ben tipizzati a compile-time.
- Rischio di Inusabilità (Restrizione): Se una regola è resa troppo restrittiva, rende il sistema meno utilizzabile.
- > Conseguenza: I programmi buoni (quelli che sono sicuri e corretti) vengono erroneamente rifiutati.

Comments

- The typing rules use very concise notation
- They are very carefully constructed
- Virtually any change in a rule either:
 - Makes the type system unsound
(bad programs are accepted as well typed)
 - Or, makes the type system less usable
(good programs are rejected)
- But some good programs will be rejected anyway
 - The notion of a good program is undecidable
(we will study this when we will talk about "computability theory")

Ogni regola è costruita con estrema cura per garantire la correttezza formale del sistema.

Il Limite della Tipizzazione: L'Indecidibilità
Nonostante l'accuratezza, alcuni programmi buoni saranno comunque rifiutati. Questo è dovuto a un limite fondamentale della teoria della computazione:

- Il Concetto di "Programma Buono" è Indecidibile: Non è possibile creare un sistema di tipi che possa decidere in modo universale ed in ogni caso se un programma è totalmente privo di errori di tipo e si comporta sempre come previsto.
- Motivazione (Teoria della Computabilità): Questa limitazione è collegata al Problema della Terminazione (Halting Problem). I sistemi di tipi devono essere conservativi: se c'è un dubbio sulla sicurezza del codice, è meglio rifiutarlo per mantenere la Soundness (sicurezza).